CSD415 PROJECT PHASE – I

DESIGN DOCUMENT – PART I

PROJECT TITLE

Behavioral Biometric-Based Continuous Authentication System for Secure Mobile Banking Using Machine Learning

Submitted by

Aiswarya R	IDK23CS006
Niranjana K D	IDK23CS045
Vimal Vijayan	IDK23CS063
Vishnu P S	IDK23CS064

Project Guide

Dr. Rani Koshy

GOVERNMENT ENGINEERING COLLEGE, IDUKKI

Department of Computer Science and Engineering

Academic Year 2026–2027

CONTENTS

1 System Design **1**

1.1 Introduction . . . 1
1.2 Design Objectives . . . 1
1.3 System Overview . . . 2
1.4 System Architecture . . . 4
1.4.1 Architectural Components . . . 5
1.4.2 Data Flow . . . 9
1.5 Data Design . . . 11
1.5.1 Data Sources . . . 12
1.5.2 Dataset Description . . . 13
1.5.3 Data Attributes and Features . . . 13
1.5.4 Data Preprocessing . . . 14
1.5.5 Data Splitting . . . 15
1.5.6 Siamese Input Formation, Pair Representation and Other Model Inputs . . . 16
1.5.7 Data Storage, Privacy and Constraints . . . 17
1.6 Machine Learning Model Design . . . 18
1.6.1 ML Problem Formulation . . . 18
1.6.2 Feature Engineering . . . 19
1.6.3 Model Selection . . . 23
1.6.4 Model Architecture . . . 24
1.6.5 Training Design . . . 26
1.6.6 Hyperparameter Design . . . 28
1.6.7 Model Evaluation Metrics . . . 29

LIST OF FIGURES

1.1 High-level system overview of the behavioral biometric-based continuous authentication system, showing the Data Acquisition, Data Processing, Authentication and Decision layers. 3

1.2 Overall input-to-output flow of the proposed system. 4

1.3 Detailed system architecture of the proposed continuous authentication system. Solid arrows represent the runtime path, while dashed enclosures and boxes identify proposed components. Matching badges indicate connections omitted from the main drawing to avoid crossing lines. K1 represents collection-mode data transfer to the dataset store; K2 represents stored user profiles and models; K3 represents shared preprocessing and feature definitions; and K4 represents the REST API connection to the backend machine learning engine. Arrow identifiers are described in Table 1.4. 6

1.4 Data flow and machine learning pipeline of the proposed continuous authentication system. The runtime path transforms raw behavioural data into a fused representation, model outputs, a trust score and a session decision. The offline path supports dataset preparation, training, enrollment and evaluation. The processing definitions are intended to remain consistent between training and runtime authentication. 11

1.5 Data preparation and model input formation for the SNN, AE and IF. Scaling parameters used in preprocessing are fitted on the training partition only (Section 1.5.4). Pair construction applies to the SNN only and is confined to the training partition; runtime samples carry no genuine/impostor label. . 12

1.6 Partitioning for the two evaluation objectives. Each objective is evaluated separately. 16

1.7 Detailed feature extraction pipeline: preprocessing, per-modality feature extraction, normalization and feature-level fusion before the three ML models. 20

1.8 Feature extraction in detail: each modality turns the preprocessed data of one window into derived quantities, summarizes them statistically, and outputs a fixed-length modality vector. 21

1.9 Normalization and feature-level fusion in detail: the four modality vectors of the same window are standardized with stored training statistics, checked for availability, concatenated in a fixed order and supplied unchanged to the three models. 22

LIST OF TABLES

1.1 Objective-to-Design Mapping 2
1.2 Architectural components: identification and purpose. 7
1.3 Architectural components: inputs, processing, outputs and interactions. 7
1.4 Connections in Figure 1.3: data carried and purpose. 8
1.5 Data flow between components; arrow identifiers refer to Figure 1.3. 10
1.6 Data source description. 13
1.7 Logical schema of raw behavioral observations. 13
1.8 Roles of behavioral samples. 14
1.9 Candidate feature groups and their required raw fields. 14
1.10 Preprocessing and quality-validation operations. 15
1.11 Dataset partitions and their intended use. 16
1.12 Distinct concepts used in the data and model interface. 17
1.13 Logical representation of an offline Siamese pair. 17
1.14 Logical storage categories. 18
1.15 Hyperparameter design (all values TBD). 28

CHAPTER 1

SYSTEM DESIGN

1.1 Introduction

This chapter describes the design of a multimodal behavioral biometric-based continuous authentication system for secure mobile banking. The system verifies, throughout an active banking session, that the person operating the application is the legitimate user. Verification begins after the initial login (PIN/fingerprint), so that unauthorized use of an unlocked, stolen or hijacked session can be detected.

The design is organized into four functional layers: Data Acquisition, Data Processing, Authentication and Decision. It is supported by a mobile banking application prototype and a REST API. Behavioral biometric data is captured passively from keystroke dynamics, touch interactions, swipe gestures, and accelerometer and gyroscope readings. Each modality is preprocessed and converted into behavioral features. These features are fused and compared against the user's enrolled behavioral profile. The Authentication layer uses machine-learning models (a Siamese Neural Network, Isolation Forest and an Autoencoder-based model), which learn the legitimate user's behavior and identify deviations or differences from it. The outputs of the authentication models are used in the Decision layer to support the generation of a dynamic trust score, which is updated continuously during the session.

The Decision layer uses the trust score to select one of three actions: continue the session, request step-up (secondary) authentication, or terminate the session. This graduated response addresses the project objectives of passive verification, comparative model evaluation, dynamic trust management and real-time demonstration. The design approach is modular: each layer can be developed and tested independently, and the system is evaluated using FAR, FRR, EER, Precision, Recall, F1-score and authentication latency.

1.2 Design Objectives

The approved objectives are taken from Section 3 of the Extended Abstract: one overall objective and six specific objectives (O1–O6).

Overall Objective. To verify identity passively throughout a banking session using machine learning and to manage session trust dynamically without disrupting the user experience.

Design Objectives. The six specific objectives are converted below into technical design objectives (DO1–DO6).

DO1. (from O1) Multimodal dataset design. Design a data acquisition mechanism and a banking-style prototype application that capture keystroke timing, touch/swipe coordinates and pressure, and accelerometer and gyroscope readings during simulated banking interactions (login, fund transfer simulation, balance inquiry, navigation). The exact dataset size is specified in Section 1.5 (Data Design).

DO2. (from O2) Behavioral feature extraction. Design a per-modality preprocessing and feature extraction pipeline that produces structured feature vectors, including keystroke dwell/flight time, swipe velocity, pressure and duration, and accelerometer/gyroscope motion statistics.

DO3. (from O3) Comparative authentication model evaluation. Design Siamese Neural Network, Isolation Forest and Autoencoder-based models for behavioral authentication, to be evaluated under the same data and feature conditions and compared with a One-Class SVM baseline.

DO4. (from O4) Dynamic trust scoring and adaptive response. Design a trust-score mechanism that uses the outputs of the authentication process to generate a continuously updated value and maps it to three actions: continue session, request step-up authentication, or terminate session.

DO5. (from O5) Quantitative evaluation. Design an evaluation procedure that measures FAR, FRR, EER, Precision, Recall, F1-score and authentication latency for each model and for the complete pipeline.

DO6. (from O6) Prototype and integration. Design a mobile application prototype and a REST API connecting the application to the backend ML engine, to demonstrate real-time continuous authentication in a simulated banking scenario.

Objective-to-Design Mapping

Table 1.1 presents the mapping between the approved project objectives and the corresponding system components or processes.

Table 1.1. Objective-to-Design Mapping

Objective	Corresponding Design Component / Process
O1: Curate multimodal behavioral dataset	Data Acquisition Process
O2: Extract discriminative behavioral features	Data Processing / Feature Extraction
O3: Comparative evaluation of anomaly detection models	Authentication Layer
O4: Dynamic trust scoring and adaptive response	Decision Layer
O5: Performance evaluation	Evaluation Module (offline pipeline), Authentication and Decision Layers
O6: Mobile application prototype with API	Mobile Application and REST API

1.3 System Overview

The proposed system continuously monitors the behavior of the user during an active mobile banking session and uses the observed behavioral patterns to generate a dynamic trust score for session-level authentication decisions. The system follows four layers (Data Acquisition, Data Processing, Authentication, Decision), as in the project's architecture diagram. The main inputs are behavioral signals generated during normal use of the banking application. The machine-learning component consists of a Siamese Neural Network, Isolation Forest and an Autoencoder-based model that learn the legitimate user's behavioral characteristics. The output is a session action (continue, step-up authentication or terminate). The user is not asked for repeated manual verification unless the system requests step-up authentication. Figure 1.1 shows the high-level block diagram.

Input

The primary inputs are behavioral biometric data generated while the user interacts with the banking application:

- **Keystroke dynamics:** key hold (dwell) and flight times during typing.
- **Touch interactions:** touch coordinates and pressure.
- **Swipe gestures:** swipe coordinates, velocity, pressure and duration.
- **Accelerometer and gyroscope data:** device motion readings.

The user also provides an initial login credential (PIN/fingerprint) before continuous authentication begins. An enrolled baseline behavioral profile of the legitimate user is the reference against which current behavior is compared. Data acquisition runs passively in the background and needs no deliberate action from the user.

Major Processing Stages

1. **Data Acquisition:** The mobile banking application captures raw touch, swipe, keystroke and motion-sensor data in real time.
2. **Preprocessing:** Each modality's raw stream is preprocessed separately.
3. **Behavioral Feature Extraction:** Each preprocessed stream is converted into a structured behavioral feature vector.
4. **Feature Fusion:** The per-modality features are combined into a single multimodal representation (the Fusion stage in the architecture diagram), which is the representation used by the authentication stage. Together with the preceding stages, it forms the Data Processing layer. The detailed fusion strategy is specified in the Data Design and Machine Learning Model Design sections.

Machine Learning Component

The Authentication layer evaluates the user's ongoing behavioral patterns against the enrolled behavioral profile using machine-learning models. The system learns the behavioral characteristics of the legitimate user and identifies deviations or differences from the established profile.

The project considers three approaches for continuous authentication: a Siamese Neural Network, Isolation Forest and an Autoencoder-based model. The Siamese Neural Network produces a similarity measure between behavioral representations, while Isolation Forest produces an anomaly score and the Autoencoder produces a reconstruction error. The detailed model configuration, training procedure and comparative evaluation are described in Section 1.6.

Trust Score and Decision

The Trust Score Generator uses the authentication results to derive a dynamic trust score, which is updated continuously according to the observed behavioral consistency of the user. The trust-score calculation, aggregation method and decision thresholds are specified in Section 1.6 (Machine Learning Model Design). The Adaptive Authentication Engine then selects one of three session actions:

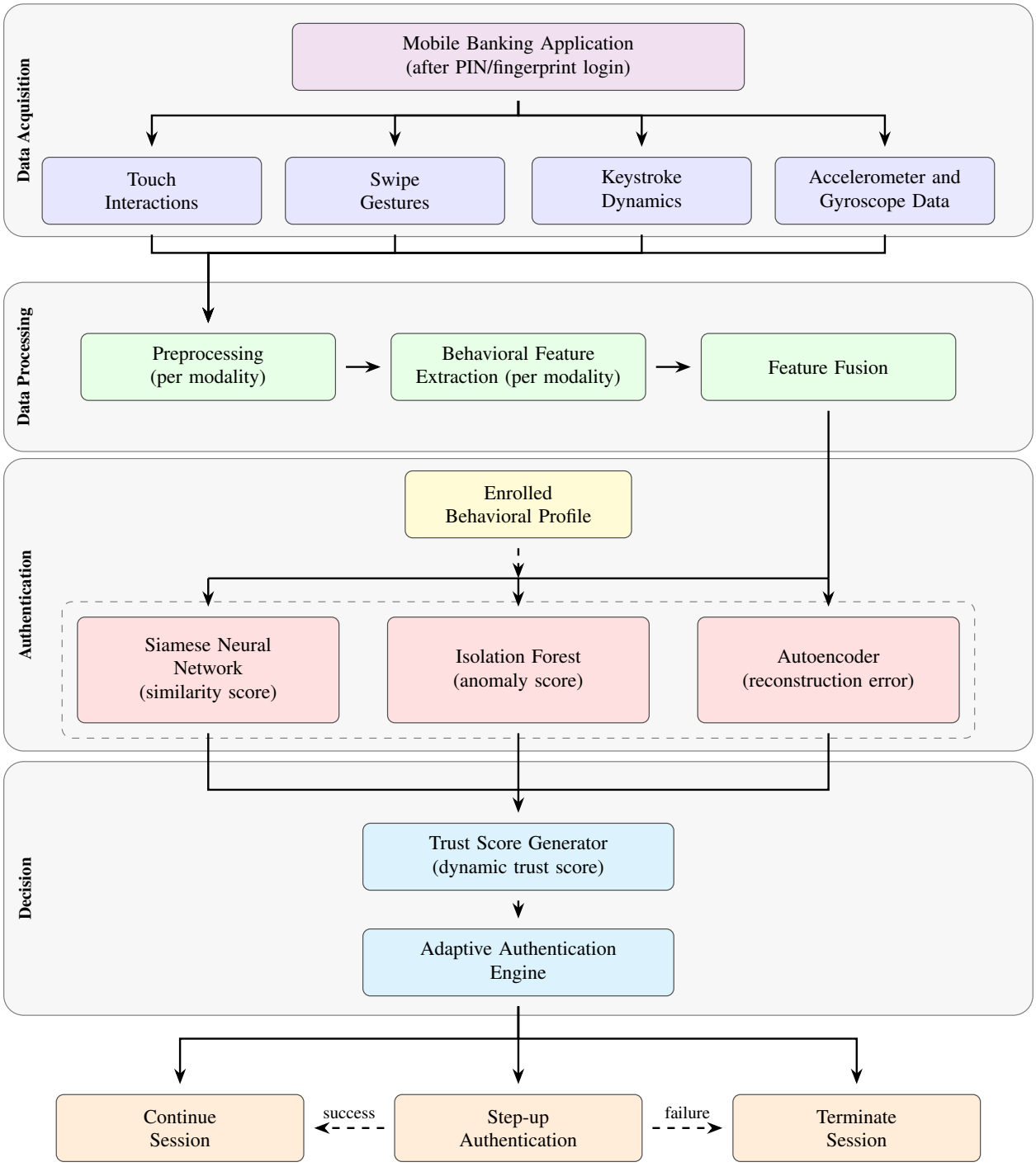


Figure 1.1. High-level system overview of the behavioral biometric-based continuous authentication system, showing the Data Acquisition, Data Processing, Authentication and Decision layers.

- **Continue session:** behavior is consistent with the legitimate user.
- **Request step-up authentication:** behavior is suspicious.
- **Terminate session:** deviation is severe, or step-up authentication fails.

Output and User Interaction

The output of the system is a session decision indicating whether the current session should continue, require step-up authentication, or be terminated. The trust score acts as an intermediate value used in making this decision. Under normal conditions, a legitimate user can continue using the mobile banking application without interruption. When the system detects significant behavioral deviation, an appropriate additional authentication response may be requested. Persistent or severe deviations may result in session termination.

The user interacts with the system through the mobile banking application prototype. The application communicates with the authentication system through the supporting backend and API components. The detailed implementation of the database, inference location, API integration and step-up authentication mechanism is specified in the subsequent

system architecture and machine-learning design sections.

Overall System Flow

After the initial login, the system continuously acquires behavioral data from the user's interaction with the mobile banking application. The data is preprocessed and converted into behavioral features, which are then combined for authentication. The authentication models evaluate the current behavioral pattern against the enrolled profile. The resulting authentication information is used to generate a dynamic trust score, which drives the adaptive authentication decision. Based on the observed behavior, the system either continues the session, requests step-up authentication, or terminates the session. If the step-up authentication succeeds, the session continues; if it fails, the session is terminated.

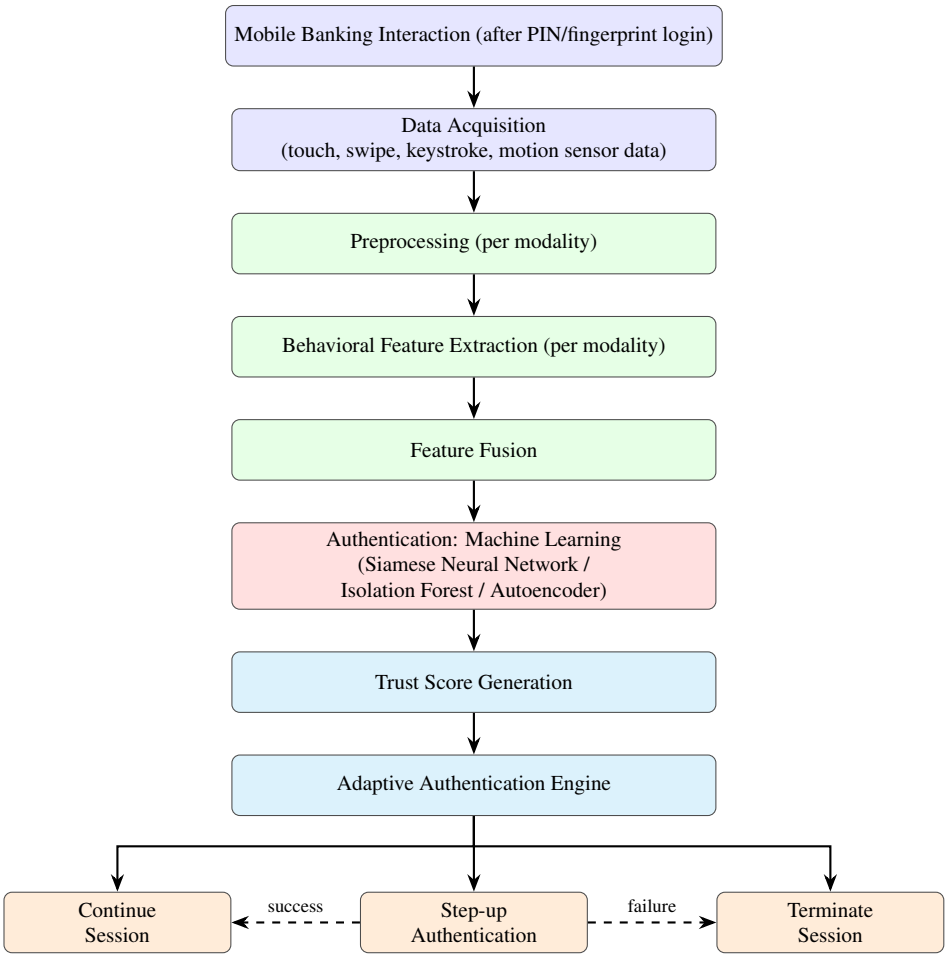


Figure 1.2. Overall input-to-output flow of the proposed system.

1.4 System Architecture

The proposed system is a layered pipeline that passively observes how a user interacts with a mobile banking application and continuously converts the observed behavior into a trust-based session decision. Figure 1.3 presents the detailed system architecture, including its components, interfaces, storage points and interactions.

The architecture consists of four principal layers: Data Acquisition, Data Processing, Authentication and Decision. These layers are supported by an Application Layer, a Data/Model Store and an offline training and evaluation pipeline. The system operates through two interconnected paths: a runtime path for continuous authentication during a live banking session and an offline path for dataset preparation, model training, enrollment and evaluation.

Application Layer.
The mobile banking application prototype provides the user interface and session handling. After the user completes the initial authentication using the existing PIN or fingerprint mechanism, the application forwards interaction events to the data acquisition layer. It applies the resulting session decisions and presents a secondary authentication challenge when required. A REST API interface is proposed to connect the application with the backend machine learning engine.

Data Acquisition Layer.
This layer captures four behavioral streams: touch dynamics, swipe patterns, keystroke dynamics and motion-sensor data obtained from the accelerometer and gyroscope. The streams are captured separately so that each

modality can undergo appropriate preprocessing and feature extraction.

Data Processing Layer.
Each behavioral stream passes through modality-specific preprocessing and behavioral feature extraction. The resulting feature vectors are combined by a fusion layer to obtain a unified behavioral representation. The precise fusion strategy and its implementation details will be finalized during the detailed system and machine learning design.

Authentication Layer.
The fused behavioral representation is analyzed using the proposed Siamese Neural Network, Isolation Forest and Autoencoder models. These models assess the relationship between the observed behavior and the enrolled user profile. Their outputs may include similarity scores, anomaly scores and reconstruction errors. These outputs provide evidence for authentication and are not, by themselves, final session decisions.

Decision Layer.
The trust score generator aggregates the model outputs into a continuously updated trust score. The adaptive authentication engine uses the resulting score to determine whether the session should continue, require secondary authentication or be terminated. As new behavioral data becomes available, the authentication process repeats throughout the active session.

Data/Model Store and Offline Pipeline.
The proposed storage components maintain the collected behavioral dataset, enrolled user profiles and trained models. The offline pipeline uses the collected data to train and evaluate the models on a development workstation. Evaluation considers False Acceptance Rate (FAR), False Rejection Rate (FRR), Equal Error Rate (EER), Precision, Recall, F1-score and inference latency. The preprocessing and feature extraction definitions used during training are intended to remain consistent with those used during runtime authentication.

The final deployment split between on-device processing and backend processing remains to be determined. In Figure 1.3, the dashed enclosure identifies the proposed backend machine learning environment. The extent of this environment may change if selected processing stages are implemented directly on the mobile device.

1.4.1 Architectural Components

The system consists of fourteen logical components (C1–C14) covering the application, behavioral data acquisition, processing, authentication, decision-making, storage, training and evaluation. Table 1.2 summarizes their responsibilities and gives the corresponding block identifiers used in Figure 1.3, while Table 1.3 describes their inputs, processing operations, outputs and interactions.

The four principal layers are Data Acquisition, Data Processing, Authentication and Decision. The application, storage and offline pipeline support these layers by providing session management, persistent data, model training and performance evaluation.

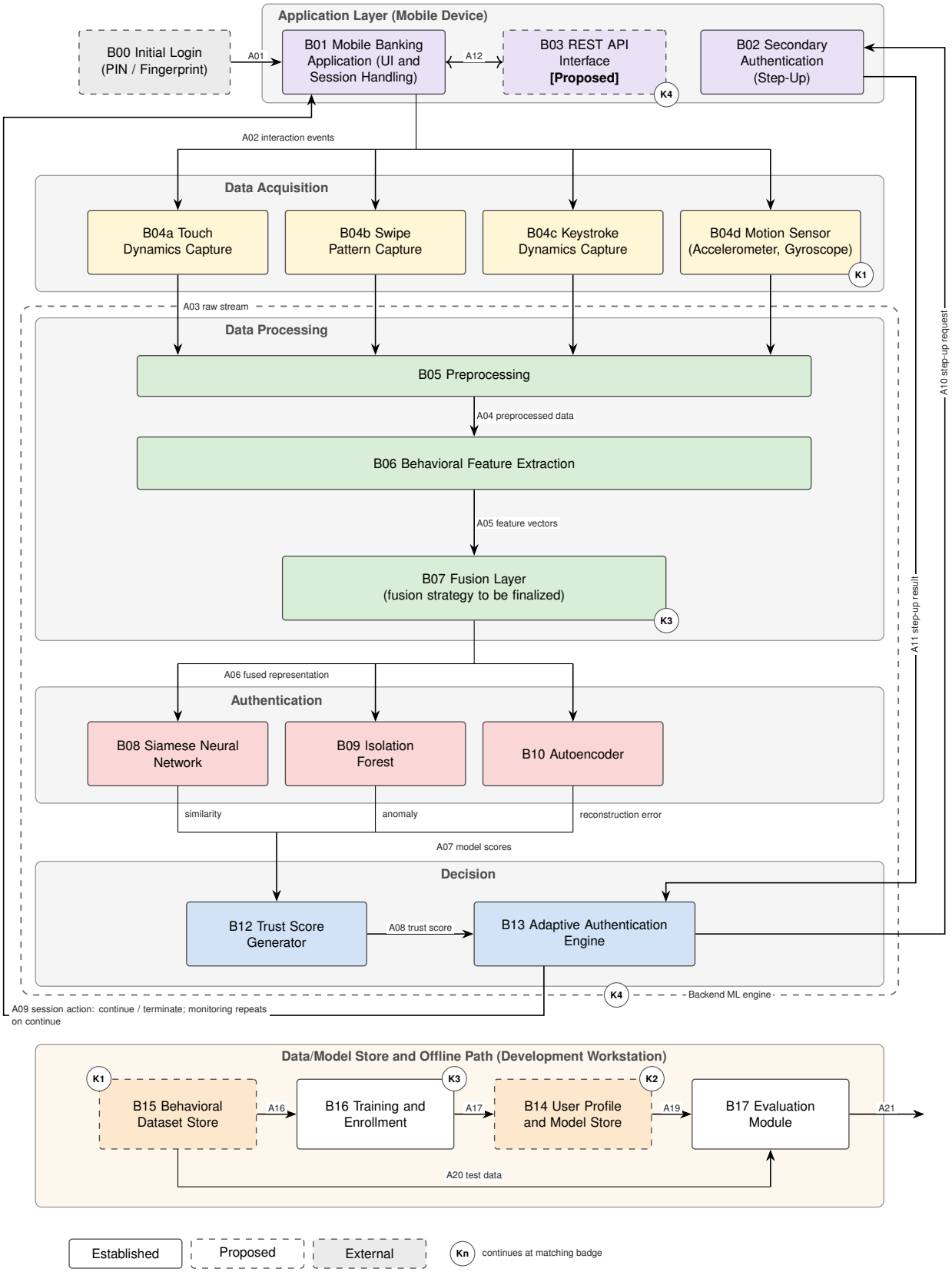


Figure 1.3. Detailed system architecture of the proposed continuous authentication system. Solid arrows represent the runtime path, while dashed enclosures and boxes identify proposed components. Matching badges indicate connections omitted from the main drawing to avoid crossing lines. K1 represents collection-mode data transfer to the dataset store; K2 represents stored user profiles and models; K3 represents shared preprocessing and feature definitions; and K4 represents the REST API connection to the backend machine learning engine. Arrow identifiers are described in Table 1.4.

Table 1.2. Architectural components: identification and purpose.

ID	Fig. 1.3	Component	Layer	Purpose
C1	B01	Mobile Banking Application Prototype	Application	Provides the banking-style interface, login, fund-transfer simulation, balance inquiry, navigation and session handling
C2	B04a–d	Data Acquisition Module	Acquisition	Passively captures raw behavioral data during interaction
C3	B05	Preprocessing Module	Processing	Prepares the raw behavioral streams from all four modalities for feature extraction
C4	B06	Behavioral Feature Extraction Module	Processing	Converts the preprocessed data into structured behavioral features
C5	B07	Fusion Layer	Processing	Combines modality-specific features into a unified representation
C6	B08–B10	Authentication Models	Authentication	Analyzes behavioral similarity and deviations from the enrolled user profile using the Siamese Neural Network, Isolation Forest and Autoencoder
C7	B12	Trust Score Generator	Decision	Aggregates model outputs into a continuously updated trust score
C8	B13	Adaptive Authentication Engine	Decision	Maps the trust score to an appropriate session action
C9	B02	Secondary Authentication Handler	Application	Requests additional authentication when required
C10	B03	REST API Interface **[Proposed]**	Application / Backend	Connects the mobile application to the backend machine learning engine
C11	B14	User Profile and Model Store **[Proposed]**	Data/Model Store	Stores enrolled user profiles and trained models
C12	B15	Behavioral Dataset Store **[Proposed]**	Data/Model Store	Maintains the collected multimodal behavioral dataset
C13	B16	Training and Enrollment Pipeline	Offline	Builds per-user profiles and trains the authentication models using legitimate-user data
C14	B17	Evaluation Module	Offline	Measures authentication performance using FAR, FRR, EER, Precision, Recall, F1-score and latency

Table 1.3. Architectural components: inputs, processing, outputs and interactions.

ID	Input	Processing	Output	Communicates with
C1	User interactions and session actions from C8	Hosts the interface and applies session decisions	Interaction events and session state	C2, C8, C9, C10
C2	Touch/swipe coordinates and pressure, key timings, accelerometer and gyroscope readings	Captures and timestamps four separate behavioral streams. Sampling rates and event formats are to be finalized	Four raw behavioral streams	C1, C3, C12
C3	Four raw behavioral streams from C2	Applies modality-specific preprocessing steps to each stream within a single module. Segmentation and windowing are proposed to support repeated feature extraction	Preprocessed data for all four modalities	C2, C4
C4	Preprocessed data from C3	Extracts features such as keystroke dwell and flight times, swipe velocity, pressure and duration, and motion statistics within a single module. The final feature list and dimensions are to be specified	Behavioral feature vectors for the four modalities	C3, C5

Continued on next page

ID	Input	Processing	Output	Communicates with
C5	Behavioral feature vectors from C4	Combines the feature vectors into a unified representation. The fusion strategy and handling of missing modalities remain to be finalized	Fused behavioral representation	C4, C6
C6	Fused representation and enrolled profile/model from C11	Uses the Siamese Neural Network, Isolation Forest and Autoencoder to obtain behavioral similarity, anomaly and reconstruction-based outputs	Model scores for trust-score generation	C5, C7, C11
C7	Model outputs from C6	Aggregates the outputs into a trust score. The aggregation formula, score scale and smoothing strategy remain to be finalized	Trust score	C6, C8
C8	Trust score and secondary-authentication result from C9	Applies the authentication thresholds to select the appropriate session action. Threshold values remain to be finalized	Continue, request secondary authentication or terminate	C7, C9, C1
C9	Secondary-authentication request from C8	Presents the additional authentication challenge. The precise challenge mechanism remains to be specified	Authentication success or failure	C8, C1
C10	Application requests and backend responses	Transports data across the application/backend boundary. The payload depends on the final deployment arrangement	Requests and responses	C1, backend ML engine
C11	Trained models and enrolled profiles from C13	Stores and provides access to per-user authentication artifacts. The storage technology remains to be finalized	Per-user profiles and trained models	C13, C6, C14
C12	Labeled behavioral sessions from collection mode	Stores and organizes the collected dataset. The storage format, dataset size and technology remain to be specified	Dataset for training and evaluation	C2, C13, C14
C13	Dataset from C12 and shared preprocessing and feature definitions	Trains the models and builds per-user enrolled profiles. The final enrollment procedure remains to be specified	Trained models and enrolled profiles	C12, C11
C14	Test data from C12 and trained models from C11	Computes FAR, FRR, EER, Precision, Recall, F1-score and latency	Evaluation results	C11, C12

The external elements of the system include the user, the mobile device and its sensors, and the existing initial authentication mechanism. The prototype focuses on the continuous authentication process rather than integration with a production banking backend.

Table 1.4 identifies the connections represented in Figure 1.3, including the data transferred and the purpose of each interaction.

Table 1.4. Connections in Figure 1.3: data carried and purpose.

Arrow	Source → Destination	Data Carried	Purpose
A01	B00 → B01	Successful-login event	Begin the monitored session
A02	B01 → B04a–d	Live touch, swipe, keystroke and device-motion events	Capture behavioral data passively

Continued on next page

Arrow	Source → Destination	Data Carried	Purpose
A03	B04a–d → B05	Four raw timestamped streams, one per modality	Prepare the data
A04	B05 → B06	Preprocessed and segmented data for all four modalities	Form inputs for feature extraction
A05	B06 → B07	Behavioral feature vectors for the four modalities	Combine the modalities
A06	B07 → B08–B10	Fused behavioral representation	Obtain model outputs
A07	B08–B10 → B12	Similarity score, anomaly score and reconstruction error	Aggregate authentication evidence
A08	B12 → B13	Trust score	Determine the session action
A09	B13 → B01	Continue or terminate action; continued sessions proceed to the next monitoring window	Apply the decision and sustain continuous authentication
A10	B13 → B02	Secondary-authentication request	Challenge the user
A11	B02 → B13	Secondary-authentication success or failure	Determine whether the session can continue
A12	B01 ↔ B03	Application requests and responses	Define the application/API boundary
A13	B03 ↔ Backend ML Engine	Data exchanged across the backend interface	Communicate with backend processing
A14	B14 → B08–B10	Enrolled user profile and trained model parameters	Provide the reference for authentication
A15	B04x → B15	Labeled raw sessions from collection mode	Build the behavioral dataset
A16	B15 → B16	Legitimate-user training samples	Train per-user models
A17	B16 → B14	Trained models and enrolled profiles	Store artifacts for runtime use
A18	B05–B07 → B16	Shared preprocessing, feature-extraction and fusion definitions	Maintain consistency between training and runtime
A19	B14 → B17	Trained models	Support evaluation
A20	B15 → B17	Test data	Evaluate authentication performance
A21	B17 → External Output	Evaluation metrics	Report system performance

1.4.2 Data Flow

Figure 1.4 illustrates how behavioral data is transformed as it passes through the system.

Runtime Data Flow. After successful initial authentication, the mobile banking application starts the monitored session and forwards user-interaction events to the data acquisition module. The module captures four behavioral streams: touch dynamics, swipe patterns, keystroke dynamics and motion-sensor data obtained from the accelerometer and gyroscope. These streams remain separate during the initial processing stages.

Each stream undergoes modality-specific preprocessing. The resulting data is segmented into windows so that behavioral features can be extracted repeatedly from the continuing interaction stream. Window length, overlap and update interval will be finalized during detailed system design.

The feature-extraction modules convert the preprocessed data into structured feature vectors. Examples include keystroke dwell and flight times, swipe velocity and pressure, and statistical features derived from accelerometer and gyroscope readings.

The modality-specific feature vectors are then combined by the fusion layer into a unified behavioral representation. This representation is analyzed by the Siamese Neural Network, Isolation Forest and Autoencoder. Their respective outputs provide information about behavioral similarity, anomalous patterns and reconstruction error.

The trust score generator aggregates the model outputs into a trust score. The adaptive authentication engine evaluates this score against the configured decision thresholds and selects one of three actions: continue the session, request secondary authentication or terminate the session.

The application applies the returned action to the active banking session. If the session continues, behavioral monitoring proceeds with subsequent data windows. This repeated process enables continuous authentication rather than

relying solely on the initial login.

Storage and Communication. The user profile and model store provides the enrolled profile and trained model artifacts required for runtime authentication. The behavioral dataset store maintains the data collected for model training and evaluation. A REST API is proposed as the communication boundary between the mobile application and the backend machine learning engine. The precise API payload and distribution of processing between the device and backend depend on the final deployment design.

Offline Training and Evaluation. Behavioral sessions collected from volunteers using the prototype application are organized into a dataset. The training and enrollment pipeline uses legitimate-user samples to develop per-user models and enrolled profiles. These artifacts are stored for subsequent runtime use.

A separate evaluation process uses test data to assess authentication performance. The evaluation considers FAR, FRR, EER, Precision, Recall, F1-score and latency. The same preprocessing and feature-extraction definitions are intended to be used during training and runtime inference to maintain a consistent behavioral representation.

Table 1.5. Data flow between components; arrow identifiers refer to Figure 1.3.

Step	Source	Data Transferred	Destination	Purpose
1	Initial Login (B00)	Session-start event (A01)	Mobile Banking Application (B01)	Begin the monitored session
2	Mobile Banking Application (B01)	Touch, swipe, keystroke and motion events (A02)	Data Acquisition (B04a–d)	Capture behavioral data
3	Data Acquisition (B04)	Four raw behavioral streams (A03)	Preprocessing (B05)	Prepare the data
4	Preprocessing (B05)	Preprocessed and segmented data (A04)	Feature Extraction (B06)	Form feature-extraction inputs
5	Feature Extraction (B06)	Four modality-specific feature vectors (A05)	Fusion Layer (B07)	Combine modalities
6	Profile and Model Store (B14)	Enrolled profiles and trained models (A14)	Authentication Models (B08–B10)	Provide user references
7	Fusion Layer (B07)	Fused behavioral representation (A06)	Authentication Models (B08–B10)	Obtain model outputs
8	Authentication Models (B08–B10)	Similarity, anomaly and reconstruction-based outputs (A07)	Trust Score Generator (B12)	Aggregate authentication evidence
9	Trust Score Generator (B12)	Trust score (A08)	Adaptive Authentication Engine (B13)	Determine the session action
10a	Adaptive Authentication Engine (B13)	Continue or terminate action (A09)	Mobile Banking Application (B01)	Apply the decision and repeat monitoring when continuing
10b	Adaptive Authentication Engine (B13)	Secondary-authentication request (A10)	Step-Up Handler (B02)	Challenge the user
11	Step-Up Handler (B02)	Authentication success or failure (A11)	Adaptive Authentication Engine (B13)	Determine whether the session can continue
12	Application and backend	Requests and responses (A12, A13)	REST API Interface (B03)	Communicate across the application/backend boundary
13	Collection-Mode Capture (B04)	Labeled raw behavioral sessions (A15)	Dataset Store (B15)	Build the dataset
14	Dataset Store (B15)	Legitimate-user training samples (A16)	Training and Enrollment (B16)	Train per-user models
15	Training and Enrollment (B16)	Trained models and enrolled profiles (A17)	Profile and Model Store (B14)	Persist runtime artifacts
16	Dataset and Model Stores (B15, B14)	Test data and trained models (A19, A20)	Evaluation Module (B17)	Measure authentication performance

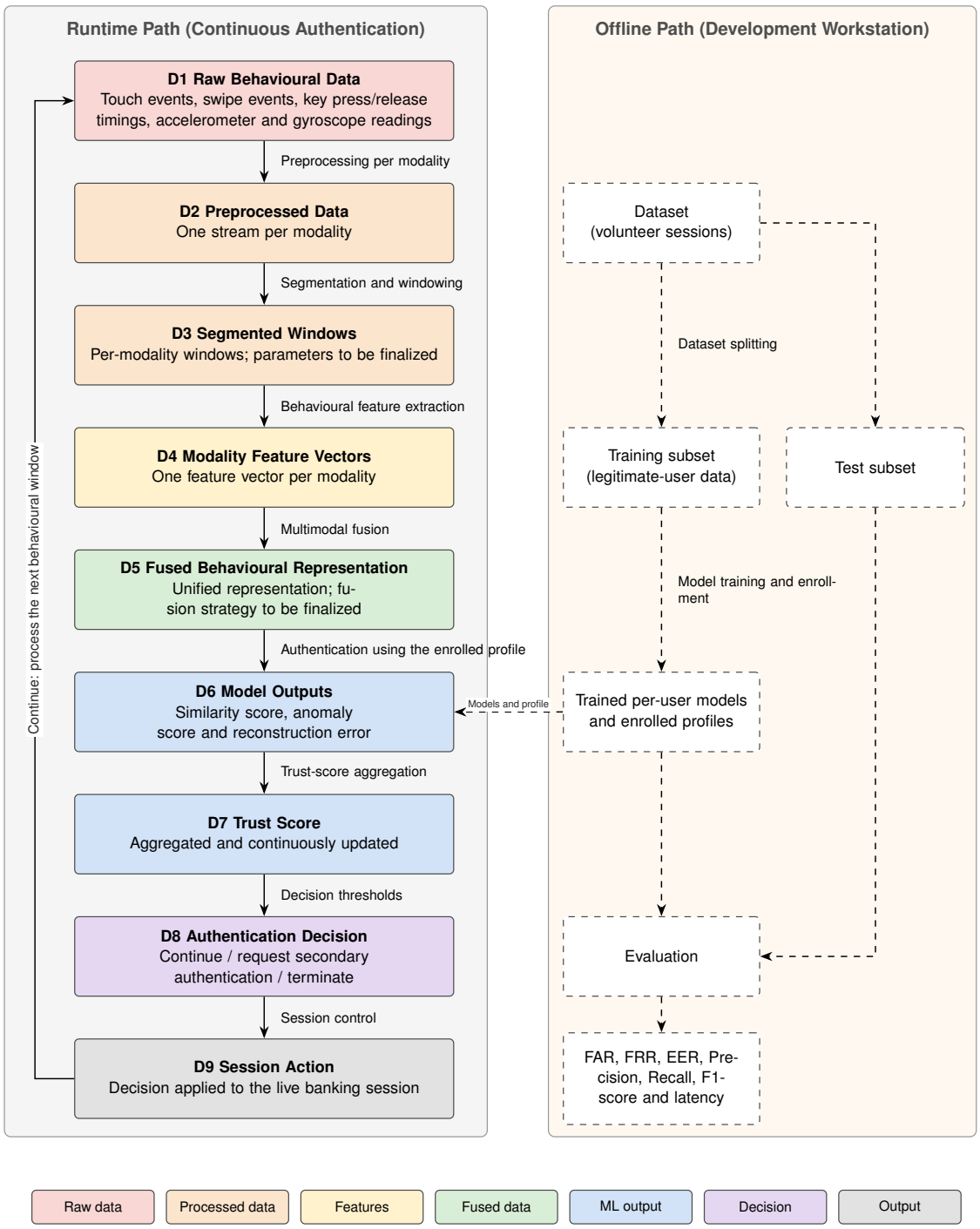


Figure 1.4. Data flow and machine learning pipeline of the proposed continuous authentication system. The runtime path transforms raw behavioural data into a fused representation, model outputs, a trust score and a session decision. The offline path supports dataset preparation, training, enrollment and evaluation. The processing definitions are intended to remain consistent between training and runtime authentication.

1.5 Data Design

This section specifies the data on which the continuous authentication system operates, how that data is represented, and how it is prepared for the three selected models: the Siamese Neural Network (SNN), the Autoencoder (AE) and the Isolation Forest (IF). The SNN compares two behavioral samples, whereas the AE and IF are one-class models that score a single sample against a profile of the legitimate user. One-Class SVM is not part of the proposed model set and is used only as a comparison baseline (Section 1.6.3); no separate data preparation is defined for it here. The section covers the data source (Section 1.5.1), the logical dataset organization (Section 1.5.2), the raw attributes and features (Section 1.5.3), preprocessing and temporal segmentation (Section 1.5.4), data splitting (Section 1.5.5), the preparation of Siamese inputs and pairs (Section 1.5.6, which also covers the inputs of the AE and IF), and storage and privacy constraints (Section 1.5.7). The model architectures, loss functions and decision logic are not defined here; they belong to Section 1.6 and Section 1.4.

The design is *proposed*; no data collection results are reported. To keep this explicit, every table carries a status marker: **C** (confirmed in the approved Extended Abstract), **S** (supported by the Literature Survey), **P** (proposed in this design, to be confirmed during implementation) and **U** (unresolved). Values not yet fixed are marked TBD. Figure 1.5 gives the overall data-preparation path used throughout the section.

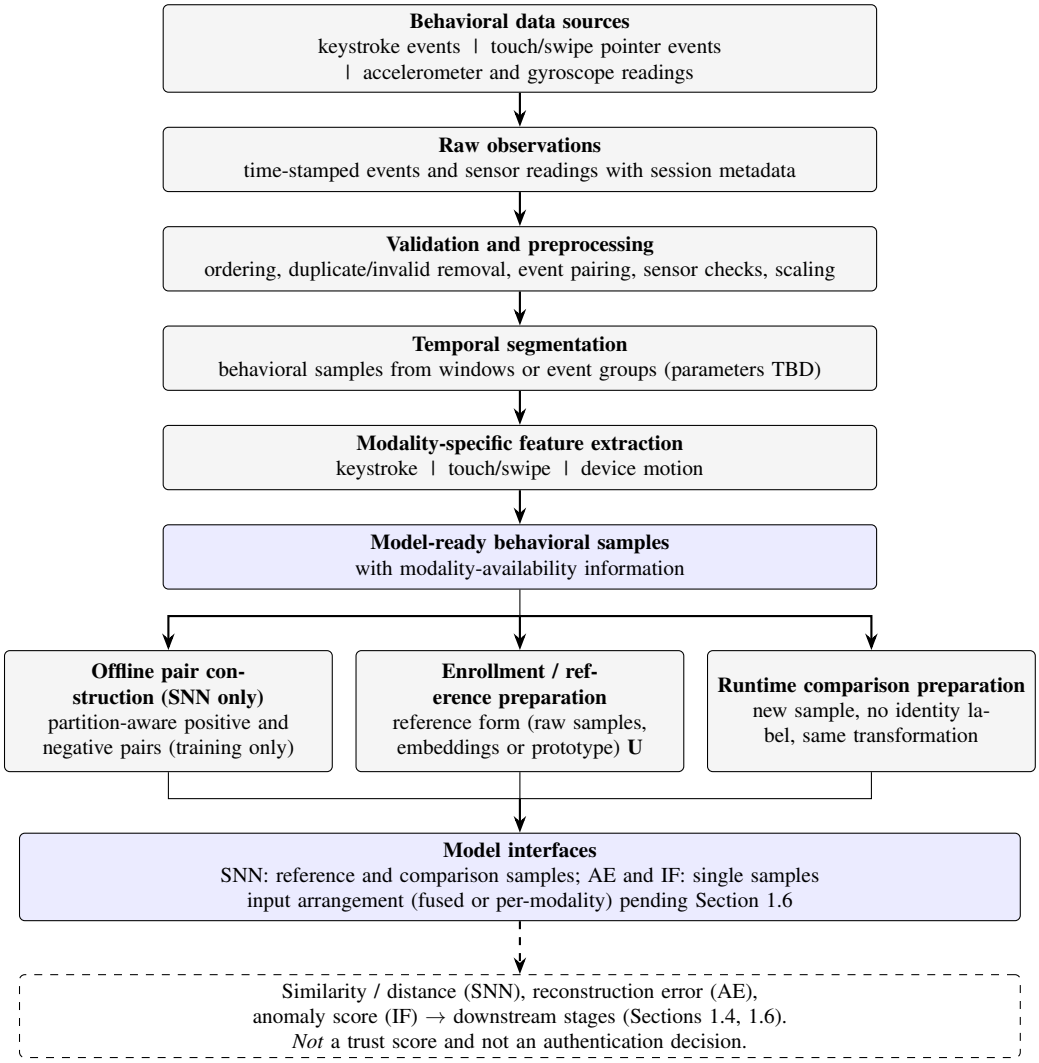


Figure 1.5. Data preparation and model input formation for the SNN, AE and IF. Scaling parameters used in preprocessing are fitted on the training partition only (Section 1.5.4). Pair construction applies to the SNN only and is confined to the training partition; runtime samples carry no genuine/impostor label.

1.5.1 Data Sources

Source and collection method. The approved Extended Abstract specifies a self-collected multimodal behavioral dataset (working title to be finalized), obtained from volunteer users interacting with a prototype banking-style application on Android smartphones. The interaction tasks are login, fund-transfer simulation, balance inquiry and navigation. Table 1.6 summarizes the data source in the format required by the design guidelines. No data collection has been completed yet, so participant, session and sample counts are left as TBD.

Behavioral modalities and their grouping. The project documents do not use one consistent count of modalities. The architecture figure of the Extended Abstract shows four acquisition streams (touch dynamics, swipe patterns, keystroke dynamics, motion sensor data); its objectives and the Literature Survey taxonomy group them as keystroke, touch/swipe, and motion (accelerometer and gyroscope); and the research-gap discussion of the Survey speaks of five signals. This section therefore uses a *working convention*: three modality groups at the data level, namely **keystroke**, **touch/swipe** and **device motion** (with separate **accelerometer** and **gyroscope** channels). Touch and swipe are treated as event types of one touchscreen pointer-event stream, since a swipe is a touch sequence with movement. At the feature level, Section 1.6.2 keeps separate touch and swipe feature blocks, which are derived from this common stream after gesture detection, so that the four-stream architecture of Section 1.4 is preserved; whether separate extractors are finally needed remains **U**. No other modality (face, voice, GPS, device fingerprint) is part of the approved scope and none is used.

Table 1.6. Data source description.

Item	Specification	Status
Dataset name	Self-collected multimodal behavioral dataset; final name TBD	C / TBD
Source type	Self-collected from volunteer participants	C
Collection setting	Prototype banking-style application: login, fund-transfer simulation, balance inquiry, navigation	C
Device platform	Android smartphones	C
Data types	Keystroke timing events; touch/swipe pointer events; accelerometer and gyroscope readings	C
Participants, sessions, samples	TBD (to be fixed after volunteer recruitment)	TBD
Labels	Pseudonymous participant identifier. No genuine/impostor label exists at collection time; pair labels for the SNN are derived offline (Section 1.5.6); the AE and IF use no labels beyond the participant identifier	P
Number of classes	Not a fixed-class problem; the number of identities equals the participant count (TBD)	P
Public data supplementation	Not decided	U

Keystroke data and credential safety. Typing behavior is represented by event timing only. The design records press and release timestamps and the task context in which they occurred, not the characters typed. Passwords, PINs, account numbers and entered amounts are never stored; the task-context code indicates only the kind of screen (for example login or transfer).

1.5.2 Dataset Description

Logical organization. The dataset is organized hierarchically: a *participant* has several *sessions*; each session contains *task segments* in which observations of the three modality groups are recorded concurrently and share one time base. This is a logical description of the required information, not a physical database design; Part II (Database Design, if applicable) is responsible for any physical realization.

Raw observation schema. Table 1.7 lists the logical fields required for raw behavioral observations. Fields marked P are proposed because the approved documents do not finalize event formats. Units, coordinate frames and sampling rates of the sensors are *not* specified in the approved material and are parameters to be confirmed during implementation.

Table 1.7. Logical schema of raw behavioral observations.

Field	Description and purpose	Logical type	Status
`participant_id`	Pseudonymous participant code; offline identity label, never a real identity	string	P
`session_id`	One continuous interaction session; unit for session-level splitting	string	P
`task_context`	Banking-style task (login, transfer simulation, balance inquiry, navigation)	categorical	C (tasks), P (coding)
`stream_type`	Keystroke, touch/swipe, accelerometer or gyroscope	categorical	P
`event_time`	Time of event or sensor reading on a single time base (time base TBC)	numeric	P
`event_type`	Press/release (keystroke); down/move/up (touch/swipe); reading (sensor)	categorical	P
`key_slot`	Non-character code used only to pair a press with its release; no typed character stored	categorical	P
`pointer_id`, `x`, `y`	Pointer identity and screen coordinates of touch/swipe events	integer, numeric	C (coordinates)
`pressure`	Touch pressure, if exposed by the device; availability TBC	numeric	C (listed), U (availability)
`axis_x`, `axis_y`, `axis_z`	Three-axis sensor values for accelerometer and gyroscope; unit and frame TBC	numeric	C (readings), U (units)
`device_descriptor`	Device model and OS version, since sensor characteristics may vary across devices	string	P
`quality_flags`	Outcome of validation checks (Section 1.5.4) for each record	categorical	P

Data roles. The same recorded data plays different roles, and only offline data has identity labels (Table 1.8). A runtime observation arrives without a genuine/impostor label; the system never knows the true identity of a live sample.

1.5.3 Data Attributes and Features

Raw attributes (Table 1.7) are converted into numerical features that describe behavior. Table 1.9 lists the candidate feature groups, the raw fields each requires and the behavioral property each represents. The features named for keystroke hold/flight time, swipe velocity, pressure and duration, and accelerometer/gyroscope patterns follow the Extended Abstract; their exact formulation (for example, which flight-time definition is used) and the final feature list,

Table 1.8. Roles of behavioral samples.

Role	Origin and identity information	Use
Enrollment/reference samples	Collected from a user at enrollment; identity known by construction	Form the reference against which later samples are compared
Offline training samples	Collected dataset; participant label available offline	Pair construction and model training
Validation samples	Held-out collected data; labeled offline	Model selection and tuning
Test samples (probes)	Held-out collected data; labeled offline	Final evaluation only
Runtime observations	Live session; *no* identity label	Compared with the reference of the claimed user

count and dimension are **U** and are to be agreed jointly with Section 1.6.2. A feature is retained only if the raw fields it needs are actually collected.

Table 1.9. Candidate feature groups and their required raw fields.

Group	Feature	Raw fields; behavioral property	Kind	Status
Keystroke	Hold (dwell) time	Press and release times of one key; how long a key is held	Derived	C
	Flight / inter-key interval	Times of consecutive key events; typing rhythm (release-to-press or press-to-press, TBC)	Derived	C (flight), U (definition)
	Timing statistics	Hold and interval values in a sample; central tendency and spread of rhythm	Statistical	P
Touch/swipe	Touch and gesture duration	Down and up times; speed and deliberateness of a gesture	Derived	C
	Start/end position, displacement	x, y at down and up; where and how far a gesture travels	Derived	C
	Speed and direction	Displacement and duration; velocity and direction of movement	Derived	C (velocity), P (direction)
	Pressure	`pressure`; touch force, if available	Raw, derived	C, U (availability)
	Trajectory summary	Sequence of x, y points; path shape relative to the straight line	Statistical	P
Device motion	Per-axis summaries	Axis values of each sensor in a window; typical posture and movement of the device	Statistical	P
	Magnitude and variation	Combined axis values over time; overall movement intensity and its variability	Derived, statistical	P

Two representations of a behavioral sample are possible: a fixed-length feature vector, or a short sequence of event-level feature values. The Extended Abstract speaks of structured feature vectors, and the Isolation Forest operates on fixed-length tabular vectors, so a fixed-length vector per behavioral sample is the proposed common representation for all three models. A sequence form could serve only the SNN and AE and would require a separate representation for the IF. The final choice, and the vector dimension, are **U** and left to Section 1.6.

1.5.4 Data Preprocessing

Table 1.10 lists the preprocessing operations required to turn raw observations into valid behavioral samples. All operations are proposed designs; the approved documents do not define a preprocessing algorithm.

Normalization without leakage. One preprocessing pipeline is shared by the three models so that their results are comparable. The scaling method (global or user-specific) is **U** and is to be agreed with Section 1.6. Whichever is chosen, parameters are estimated on training data only, stored as part of the preprocessing artifact, and reused unchanged for validation, test, enrollment and runtime data. If user-specific scaling is selected, its statistics may be computed only from that user's enrollment samples, never from test or runtime observations.

Temporal segmentation. Continuous observations are organized into behavioral samples that the SNN can compare. Four strategies are possible: *event-based* samples (one gesture or key event), *fixed-duration* windows, *sliding* (overlapping) windows, and *sufficiency-based* windows that close once a minimum number of events is collected. Keystroke and swipe events are sparse, whereas motion data is dense, so a single fixed duration may give some samples too few keystrokes and others too few swipes. The design therefore proposes that a behavioral sample be a time-aligned container holding the segment of each modality for the same interval, together with the availability indicator above. Window duration or event count, overlap, minimum usable information, and the handling of missing modalities are **U**. The feature-extraction pipeline of Section 1.6.2 assumes fixed windows for illustration only; the final strategy is fixed here and not there. Section 1.6 and the authentication interval defined in Section 1.4 depend on these values, and no values are fixed in this document. Overlapping windows create near-duplicate samples; the consequences for splitting are handled in Section 1.5.5.

Table 1.10. Preprocessing and quality-validation operations.

Operation	Input and processing	Purpose	Output	Status
Timestamp validation and ordering	Raw records; reject invalid timestamps, sort chronologically within a stream	Timing features depend on correct order	Ordered records	P
Duplicate, malformed and invalid-value handling	Ordered records; remove duplicates, flag malformed records and non-finite or implausible values	Prevent corrupt values from distorting features	Validated records with quality flags	P
Keystroke event pairing	Validated key events; match each press with its release via `key_slot`; flag unmatched events	Hold time needs both events; flight time needs consecutive pairs	Complete key events; incomplete ones flagged	P
Touch/swipe grouping	Validated pointer events; group down–move–up sequences into gestures (taps and swipes); flag incomplete gestures; criterion separating tap from swipe TBC	Gesture features need complete gestures	Gesture records	P
Sensor validation and alignment	Accelerometer and gyroscope readings; check gaps and irregular intervals; relate readings to the common time base; no resampling or interpolation algorithm selected	Sensors may be irregular and not perfectly synchronized	Validated, time-aligned readings	P, U (method)
Noise handling	Validated sensor readings and event intervals; optional smoothing filter and cap on abnormal pauses (Section 1.6.2)	Limit the influence of sensor noise and long idle gaps	Smoothed readings and intervals	P, U (method)
Scaling / normalization	Feature values; fit scaling parameters on the training partition only; apply the stored transformation to validation, test, enrollment and runtime data	Features have different ranges, and unscaled inputs hamper neural-network training	Scaled features	P, U (method, scope)
Modality availability marking	Per-sample modality presence; record which modalities have enough usable data	Typing and swiping do not occur continuously	Availability indicator per sample	P

1.5.5 Data Splitting

The evaluation objective is not yet fixed in the approved documents, so two distinct objectives are described and must not be mixed. Table 1.11 and Figure 1.6 show their organization. Split ratios are not fixed; they depend on the participant and session counts (TBD).

Objective A: personalized authentication. The system holds a user's enrollment samples and evaluates later behavior of that user against them. The same participants therefore appear in all partitions, so the split must be made by *session*, never by window. Each participant's sessions are assigned to training, enrollment, validation and test roles, with test sessions later in time than enrollment sessions (proposed). Enrollment samples serve only as references and are never used as probes. This avoids comparing samples with themselves or with overlapping near-duplicates.

Objective B: generalization to unseen participants. To test identities not seen in training, the split is made by *participant*: training, validation and test participants are disjoint. For a test participant, early sessions supply enrollment references and later sessions supply probes. Preprocessing parameters and the network are fitted without any test participant. A session-level split does not evaluate this objective, and a participant-level split does not evaluate Objective A; each must be reported separately. If the number of volunteers is small, grouped cross-validation by participant is a proposed alternative.

Genuine-only fitting of the AE and IF. The Extended Abstract describes per-user one-class models. The AE and IF are therefore fitted on the genuine user's samples only; impostor samples are never used in fitting, and are used only as probes for evaluation and for tuning on the validation partition. Under Objective A, a user's AE and IF are fitted on that user's training or enrollment sessions and evaluated on held-out sessions. Under Objective B, they are fitted at enrollment from the held-out participant's early sessions, so no test-participant data is needed for fitting; only shared hyperparameters are chosen on training and validation participants. Whether the models are trained per user or once globally (the SNN is a shared model in either case) is **U**. For a fair comparison, all three models use the same

Table 1.11. Dataset partitions and their intended use.

Partition	Objective A (session-level)	Objective B (participant-level)
Training	Training sessions of all participants; used for fitting preprocessing, SNN pair construction and training; genuine data for AE/IF fitting	All sessions of training participants only
Enrollment	Enrollment sessions of each participant; references only	Early sessions of validation/test participants; references only
Validation	Validation-probe sessions; model selection and tuning	Validation participants; model selection and tuning
Test	Test-probe sessions, later than enrollment; used once for final evaluation	Later sessions of test participants; used once

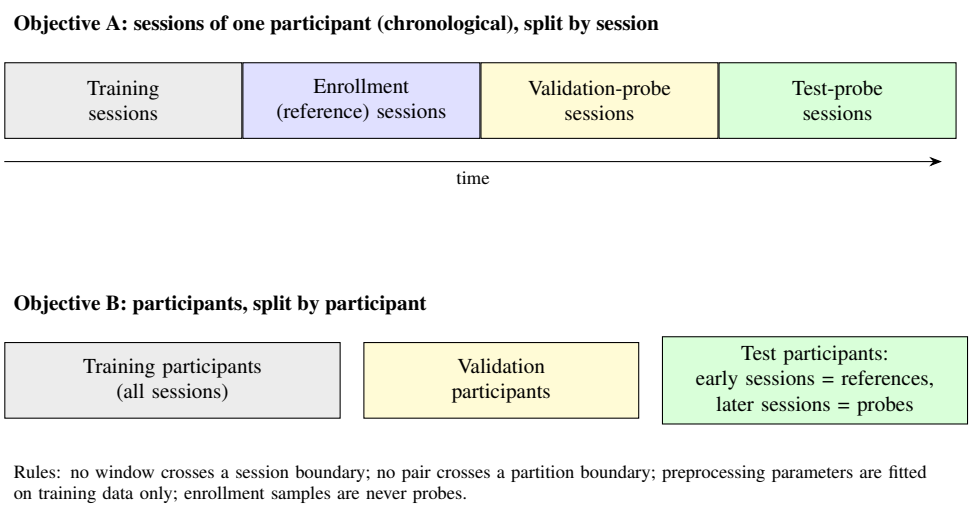


Figure 1.6. Partitioning for the two evaluation objectives. Each objective is evaluated separately.

partitions, the same probes and the same preprocessing parameters.

Leakage prevention. The following rules apply to both objectives and are proposed for adoption:

- Partition assignment is done before windowing, at session level (A) or participant level (B), so overlapping windows from one session never lie in two partitions.
- Preprocessing parameters, thresholds and hyperparameters are fitted or tuned using training and validation data only; test data are used once for final reporting.
- Training pairs for the SNN are generated after partitioning and only from training samples, so a test sample can never enter a training pair, directly or indirectly (Section 1.5.6).
- Duplicate and near-duplicate samples are removed or grouped, so that two heavily overlapping windows never form a positive pair or fall on opposite sides of a split.
- Impostor samples are never used to fit the AE or IF, and no probe is used to fit any model.
- Enrollment samples are excluded from the probe set; when one reference set is reused across many probes, each probe is still scored independently.

1.5.6 Siamese Input Formation, Pair Representation and Other Model Inputs

Inputs of the Autoencoder and Isolation Forest. The AE and IF consume one model-ready sample at a time and need no pair records or pair labels. The AE is fitted on genuine samples and outputs a reconstruction error; the IF is fitted on genuine samples and outputs an anomaly score. Both therefore depend on the fixed-length vector representation of Section 1.5.3. The remainder of this subsection concerns the SNN, which alone requires pairs.

Pair-comparison concept. A Siamese network processes two inputs through branches that share the same learned parameters, so that it learns a representation in which behavioral similarity can be measured; a single shared network of this kind has been used for keystroke verification so that a new user can be enrolled from a handful of samples , and keystroke embeddings learned with contrastive and triplet losses have been studied at scale . At the data-design level, the network receives a *reference* sample and a *comparison* sample, each as a model-ready representation, and returns similarity or distance information. How the two inputs are fused or kept modality-specific is decided in Section 1.6. The terms used in this report are kept distinct as shown in Table 1.12.

Pair records. Pairs reference existing samples by identifier and do not copy raw data (Table 1.13). These are logical fields only.

Table 1.12. Distinct concepts used in the data and model interface.

Concept	Meaning	Defined in
Participant identity	Pseudonymous owner of the data; offline label only	1.5.2
Sample identity	Identifier of one behavioral sample	1.5.6
Pair relationship label	Same-identity or different-identity label of an offline pair; not an authentication outcome	1.5.6
Input representation	Model-ready form of a sample	1.5.3, 1.5.4
Learned embedding	Representation produced inside the network, if used	1.6
Similarity / distance	SNN output comparing reference and comparison samples	1.6
Reconstruction error, anomaly score	AE and IF outputs for a single sample; model scores, not decisions	1.6
Trust score	Session-level value derived downstream; not defined here	1.4, 1.6
Authentication decision, session action	Continue, step-up or terminate, as in the approved workflow	1.4

Table 1.13. Logical representation of an offline Siamese pair.

Field	Description	Status
`pair_id`	Identifier of the pair	P
`sample_a_ref`, `sample_b_ref`	References to the two processed samples	P
`identity_relation`	Same participant or different participants (offline only)	P
`pair_label`	Label required by the chosen objective, if any (for example same/different)	U
`partition`	Training, validation or test partition of both samples; both must match	P
`session_relation`	Whether the samples come from the same or different sessions	P

Positive and negative pairs. Under the conventional pairing, a *positive pair* contains two samples of the same participant and a *negative pair* two samples of different participants. This framing depends on the training objective, which is **U**; contrastive, triplet and binary objectives are possible and are decided in Section 1.6. A triplet objective would replace pair records by (anchor, positive, negative) records with the same rules. Negative pairs require data of other volunteers, so they are available only during offline model development (the AE and IF do not need them); at runtime only the enrolled user's own samples are needed as reference. The proposed pair-generation rules are:

- generate pairs within one partition only, after partitioning;
- form positive pairs from non-overlapping samples of one participant, preferably from different sessions, and never pair a sample with itself or its overlapping neighbor;
- form negative pairs from different participants, preferably within the same task context, so the network does not learn task differences instead of identity;
- remove duplicate pairs; the positive-to-negative ratio, the sampling strategy and the number of pairs are TBD.

The pair label is used only during offline development; it is distinct from any authentication outcome.

Reference and comparison flow. The proposed flow is: (1) collect and prepare enrollment samples; (2) construct the reference representation required by the model design; (3) collect and preprocess a new runtime sample using the stored transformation; (4) form its model-ready representation; (5) compare it with the reference through the network; and (6) pass the similarity or distance information to the downstream decision stage. For the AE and IF, the enrollment samples are instead the fitting data of the user's one-class model, and the stored profile is the fitted model. The form in which the SNN reference is kept (raw samples, stored embeddings, a prototype, or several references) and the number of enrollment samples are **U** and require coordination with Section 1.6. Automatic profile updating and online retraining are not part of the approved documents and are not assumed here.

1.5.7 Data Storage, Privacy and Constraints

Logical storage requirements. Table 1.14 lists the information categories the design needs. No physical database, file format, cloud service or deployment technology is selected here; the Extended Abstract lists candidate software, but no data-storage decision is adopted in this section. Retention and handling are proposed.

Consent, privacy and data minimization. The following safeguards are proposed and are not yet implemented. Volunteers give informed consent before collection, stating what is recorded and for what purpose. Participants are identified only by pseudonymous codes, and any table linking codes to real persons is kept separately with restricted access. Only the fields in Table 1.7 are recorded: no typed characters, passwords, PINs, credentials or real banking data are collected, and the banking tasks use a simulated application. Access to raw data and to enrollment information is limited to authorized team members, with a defined retention period and secure deletion afterwards, and raw data need not be retained permanently once features are verified. Compliance with any specific regulation is not claimed, because no regulatory requirement is established in the source documents; the broader treatment belongs to Section 2.8 of Part II.

Table 1.14. Logical storage categories.

Category	Content and purpose	Phase	Proposed handling
Raw observations	Time-stamped events and readings (Table 1.7)	Offline	Restricted access; retained only as long as needed for feature (re)extraction
Processed samples and features	Validated, segmented and extracted samples	Offline / runtime	Retained for training and reproducibility
Participant and session metadata	Pseudonymous identifiers, session and device descriptors	Offline	Stored separately from the identity key
Pair definitions	References to samples (Table 1.13)	Offline	Regenerable from partitions
Enrollment/reference information	SNN reference in the form selected in Section 1.6; fitted per-user AE/IF profile, if per-user models are used	Runtime	Protected; access restricted to the authentication function
Model artifacts and preprocessing parameters	Trained SNN, AE and IF models, stored scaling parameters	Both	Versioned together
Evaluation results	Metrics and partition definitions	Offline	Retained with the report

Constraints. The data design is subject to the following constraints: dataset size depends on the number of volunteers (TBD); the data come from Android devices with possibly different sensors; sensor sampling rate, units and time base remain to be confirmed; and sparse keystroke and swipe activity may leave some samples without all modalities.

1.6 Machine Learning Model Design

This section formulates the machine-learning problem, describes the feature representation seen by the models, justifies the model selection, and specifies the architecture, training, hyperparameters and evaluation metrics of the three proposed models.

1.6.1 ML Problem Formulation

Problem Type

The proposed system is a multimodal continuous authentication system for mobile banking sessions. Among the standard problem types, the primary formulation is **anomaly detection**. The system learns the normal behavioral profile of the legitimate user and flags deviation from it, because behavioral samples from attackers are not available in advance.

The system also uses **similarity learning**, a metric-learning formulation. The final session decision is a thresholded mapping of a trust score, so classification-type metrics such as FAR, FRR, precision and recall are used to evaluate it. The models themselves are not trained as impostor-versus-genuine classifiers.

Regression, clustering, recommendation, forecasting, object detection, image segmentation, NLP and generative AI do not apply to this project.

Input and Feature Representation

In each session window t, the passively captured keystroke, touch, swipe and motion signals are converted into a fused behavioral feature vector

$$\mathbf{x}_t \in \mathbb{R}^d, \qquad d = \text{TBD} \tag{1.1}$$

where d is the final feature dimension, finalized after feature selection (see Sections 1.5.3 and 1.6.2).

Let $\mathscr{D}_u$ denote the enrollment data of a legitimate user u. It forms the reference behavioral profile of that user.

Model Mappings

Three models operate on $\mathbf{x}_t$, following the general form:

$$X \to f_\theta(X) \to Y \tag{1.2}$$

1. **Siamese Neural Network:**

$$s_{\text{sim}}(\mathbf{x}_t) = \text{sim}\,(f_\theta(\mathbf{x}_t), \mathbf{r}_u) \tag{1.3}$$

   The similarity between the embedding of $\mathbf{x}_t$ and the reference embedding(s) $\mathbf{r}_u$ of user u. A higher value indicates behavior more likely to be genuine.
2. **Autoencoder:**

$$s_{\text{ae}}(\mathbf{x}_t) = \left\| \mathbf{x}_t - g_\phi\left(h_\psi(\mathbf{x}_t)\right) \right\|^2 \tag{1.4}$$

The reconstruction error, where h_ψ is the encoder and g_ϕ is the decoder. A higher value indicates more anomalous behavior.

3. **Isolation Forest:**

$$s_{\text{if}}(\mathbf{x}_t) = 2^{-\frac{\mathbb{E}[h(\mathbf{x}_t)]}{c(n)}} \tag{1.5}$$

The anomaly score computed from the average path length $\mathbb{E}[h(\mathbf{x}_t)]$ over the isolation trees; $c(n)$ is the normalizing term defined in Section 1.6.4. A higher value indicates more anomalous behavior.

Authentication Output and Decision

The three scores are normalized to the range $[0,1]$, oriented to a common direction (higher = more genuine), and combined by the Trust Score Generator (the orientation and normalization steps are given in Section 1.6.4):

$$T_t = \Phi(\hat{s}_{\text{sim}}, \hat{s}_{\text{ae}}, \hat{s}_{\text{if}}) \in [0,1] \tag{1.6}$$

The Adaptive Authentication Engine then applies two thresholds $\tau_1 > \tau_2$ to the trust score (or to its temporally smoothed version $\widetilde{T}_t$, Section 1.6.4):

$$\begin{cases} T_t \geq \tau_1 & \rightarrow \text{continue session,} \\ \tau_2 \leq T_t < \tau_1 & \rightarrow \text{request step-up (secondary) authentication,} \\ T_t < \tau_2 & \rightarrow \text{terminate session.} \end{cases} \tag{1.7}$$

The three outcomes are defined in the project workflow. If step-up authentication fails, the session is terminated. The exact form of Φ, the fusion weights and the thresholds are proposed and will be fixed on validation data.

Training Time, Inference Time and Continuous Operation

Training time: Feature vectors from the self-collected dataset are used to learn the Siamese embedding from sample pairs, to fit the Autoencoder on normal behavior, and to build the Isolation Forest trees. Score-normalization statistics and the thresholds τ_1 and τ_2 are calibrated on validation data.

Enrollment and inference: The legitimate user's enrollment samples are used to build the reference embedding(s) $\mathbf{r}_u$ and baseline score statistics. For each new window, the trained models produce s_{sim}, s_{ae} and s_{if}.

Continuous authentication: The scoring, fusion and decision steps are repeated for every window until the session ends, so the trust score T_t is updated throughout the session.

Why This Is Continuous Authentication

A PIN, password or fingerprint establishes identity once, at login ($t = 0$), and says nothing about who operates the device afterwards. A stolen, hijacked or left-unlocked session therefore remains trusted until logout or timeout.

In the proposed system, the identity question is asked again for every window t using signals collected passively in the background. An unauthorized user can therefore be detected while acting, and the response is graded (continue, step-up, terminate) rather than a single accept or reject. This is also why authentication latency is included in the evaluation metrics.

1.6.2 Feature Engineering

Note on scope: The raw attributes, derived features and preprocessing steps are defined in Sections 1.5.3 and 1.5.4. This section describes only the machine-learning-facing representation of those features.

Features Used

The features come from the four behavioral streams captured by the mobile banking application prototype: keystroke dynamics, touch dynamics, swipe patterns and motion sensor data (accelerometer and gyroscope).

- **Keystroke dynamics:** key hold (dwell) time and flight time between key events.
- **Touch dynamics:** touch coordinates and pressure recorded during interaction with the application.
- **Swipe patterns:** swipe coordinates, velocity, pressure and duration.
- **Motion sensors:** accelerometer and gyroscope readings, summarized as motion statistics.

Feature Extraction Pipeline

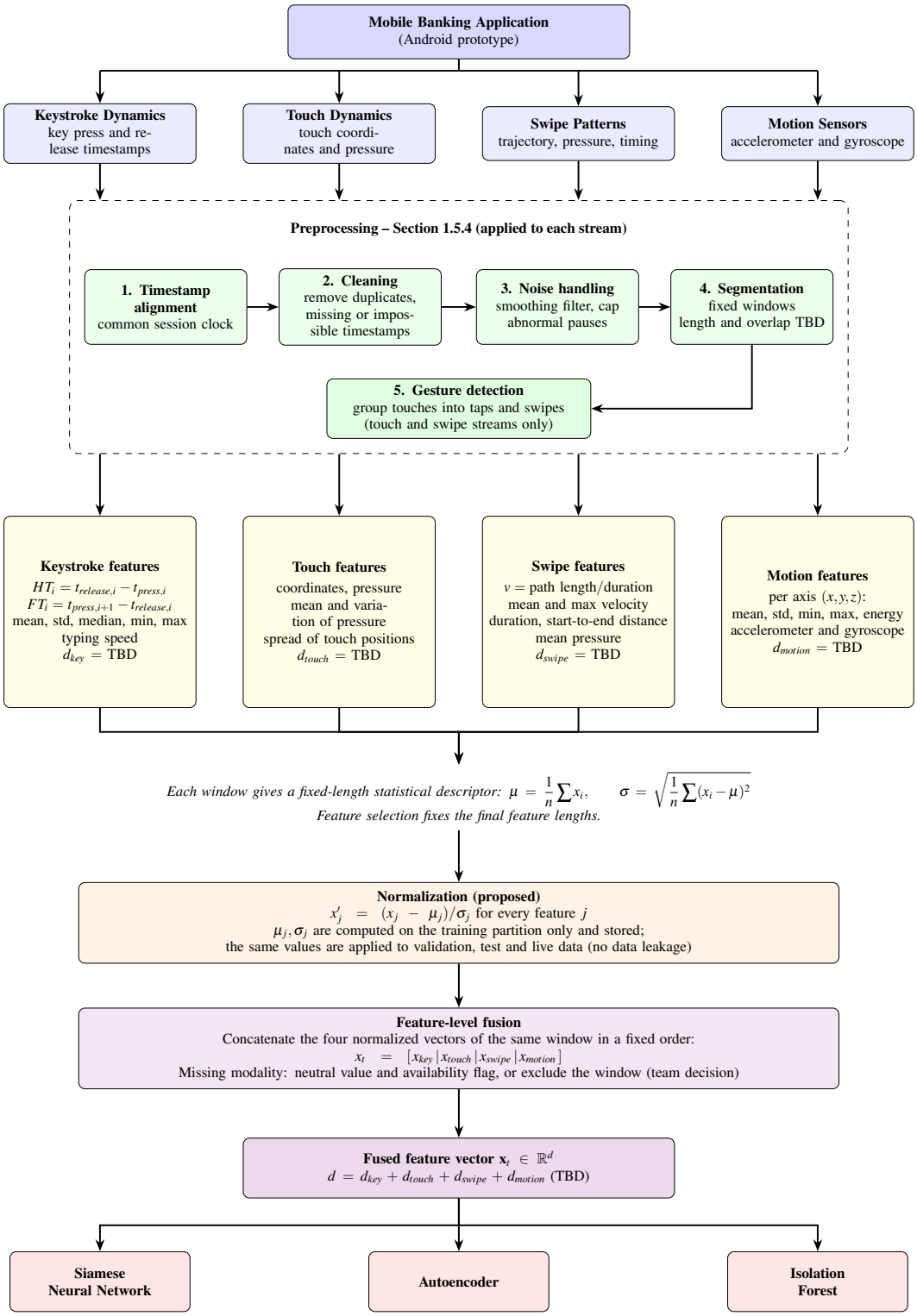


Figure 1.7. Detailed feature extraction pipeline: preprocessing, per-modality feature extraction, normalization and feature-level fusion before the three ML models.

The pipeline in Figure 1.7 converts raw signals into model-ready input in the following stages.

1. **Data capture.** The Android prototype of the mobile banking application records four behavioral streams while the user interacts with it: keystroke dynamics (key press and release timestamps), touch dynamics (touch coordinates and pressure), swipe patterns (trajectory, pressure and timing), and motion sensors (accelerometer and gyroscope readings).
2. **Preprocessing (Section 1.5.4).** Each stream passes through the following steps: (1) timestamp alignment to a common session clock; (2) cleaning, which removes duplicate events and events with missing or impossible timestamps; (3) noise handling, using a smoothing filter and capping of abnormal pauses; (4) segmentation into windows

(fixed windows are assumed in this section for the feature definitions; the final strategy, window length and overlap are to be finalized in Section 1.5.4); and (5) gesture detection, which groups touch events into taps and swipes (touch and swipe streams only).

3. **Feature extraction (Section 1.5.3).** Features are computed separately for each modality over each window, as shown in Figure 1.8:
   - *Keystroke:* hold time $HT_i = t_{release,i} - t_{press,i}$ and flight time $FT_i = t_{press,i+1} - t_{release,i}$, summarized by mean, standard deviation, median, minimum and maximum, together with typing speed (dimension d_{key}, TBD).
   - *Touch:* coordinates and pressure, summarized by the mean and variation of pressure and the spread of touch positions (dimension d_{touch}, TBD).
   - *Swipe:* velocity $v =$ path length/duration, summarized by mean and maximum velocity, duration, start-to-end distance and mean pressure (dimension d_{swipe}, TBD).
   - *Motion:* per-axis (x, y, z) mean, standard deviation, minimum, maximum and energy of the accelerometer and gyroscope signals (dimension d_{motion}, TBD).

   Each window therefore yields a fixed-length statistical descriptor, using for example

$$\mu = \frac{1}{n}\sum x_i$$

   and

$$\sigma = \sqrt{\frac{1}{n}\sum (x_i - \mu)^2}.$$

   Feature selection fixes the final feature lengths.

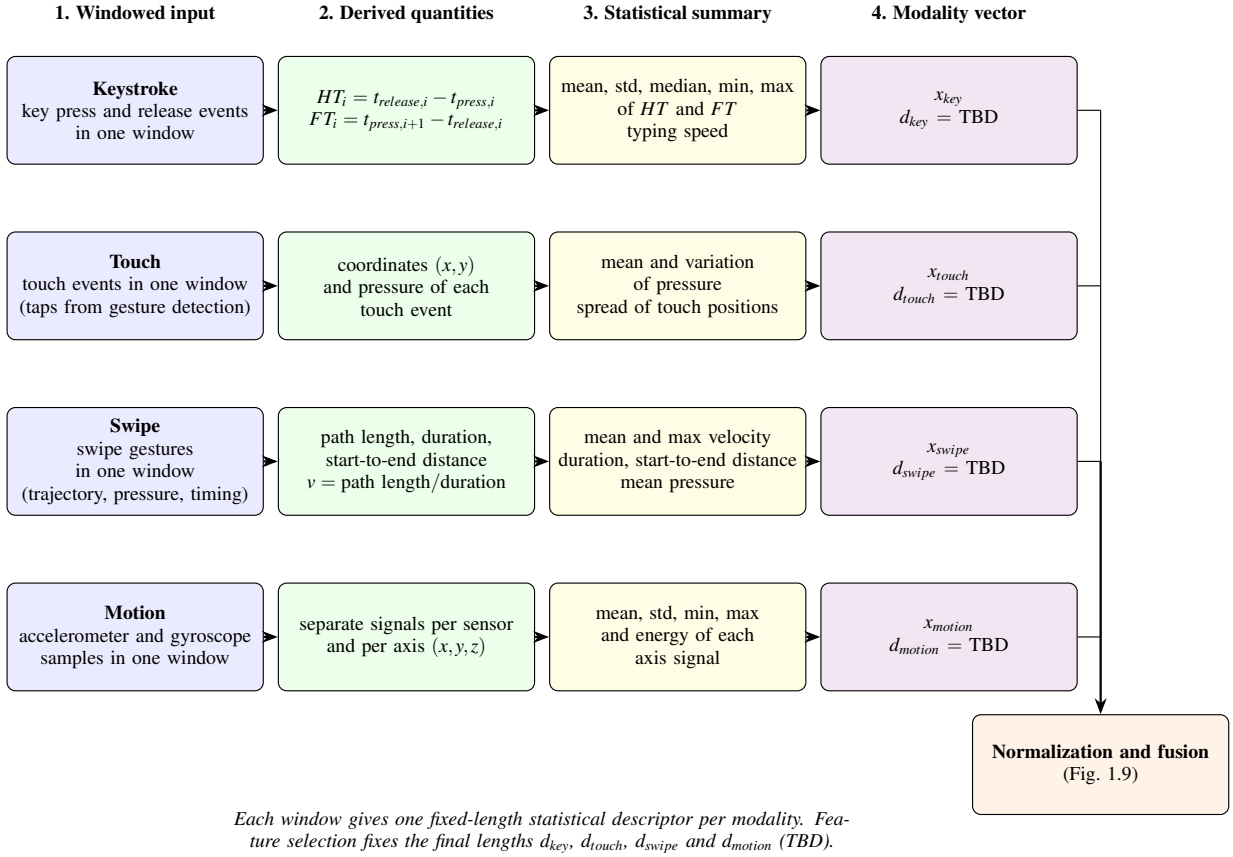


Figure 1.8. Feature extraction in detail: each modality turns the preprocessed data of one window into derived quantities, summarizes them statistically, and outputs a fixed-length modality vector.

‘‘‘

4. **Normalization (proposed).** Each feature is standardized as

$$x'_j = \frac{x_j - \mu_j}{\sigma_j},$$

   where μ_j and σ_j are computed on the training partition only and stored. The same stored values are applied to validation, test and live data, which prevents data leakage (see the Feature Normalization subsection below and Figure 1.9).

5. **Feature-level fusion.** The four normalized vectors of the same window are concatenated in a fixed order, as shown in Figure 1.9:

$$\mathbf{x}_t = [x'_{key} \,|\, x'_{touch} \,|\, x'_{swipe} \,|\, x'_{motion}] \in \mathbb{R}^d,$$

where

$$d = d_{key} + d_{touch} + d_{swipe} + d_{motion} \quad \text{(TBD)}.$$

When a modality is absent from a window, the proposed options are a neutral value with an availability flag, or exclusion of the window. This is to be decided during implementation.

6. **Model input.** The same fused vector $\mathbf{x}_t$ is supplied unchanged to the Siamese Neural Network, the Autoencoder and the Isolation Forest.

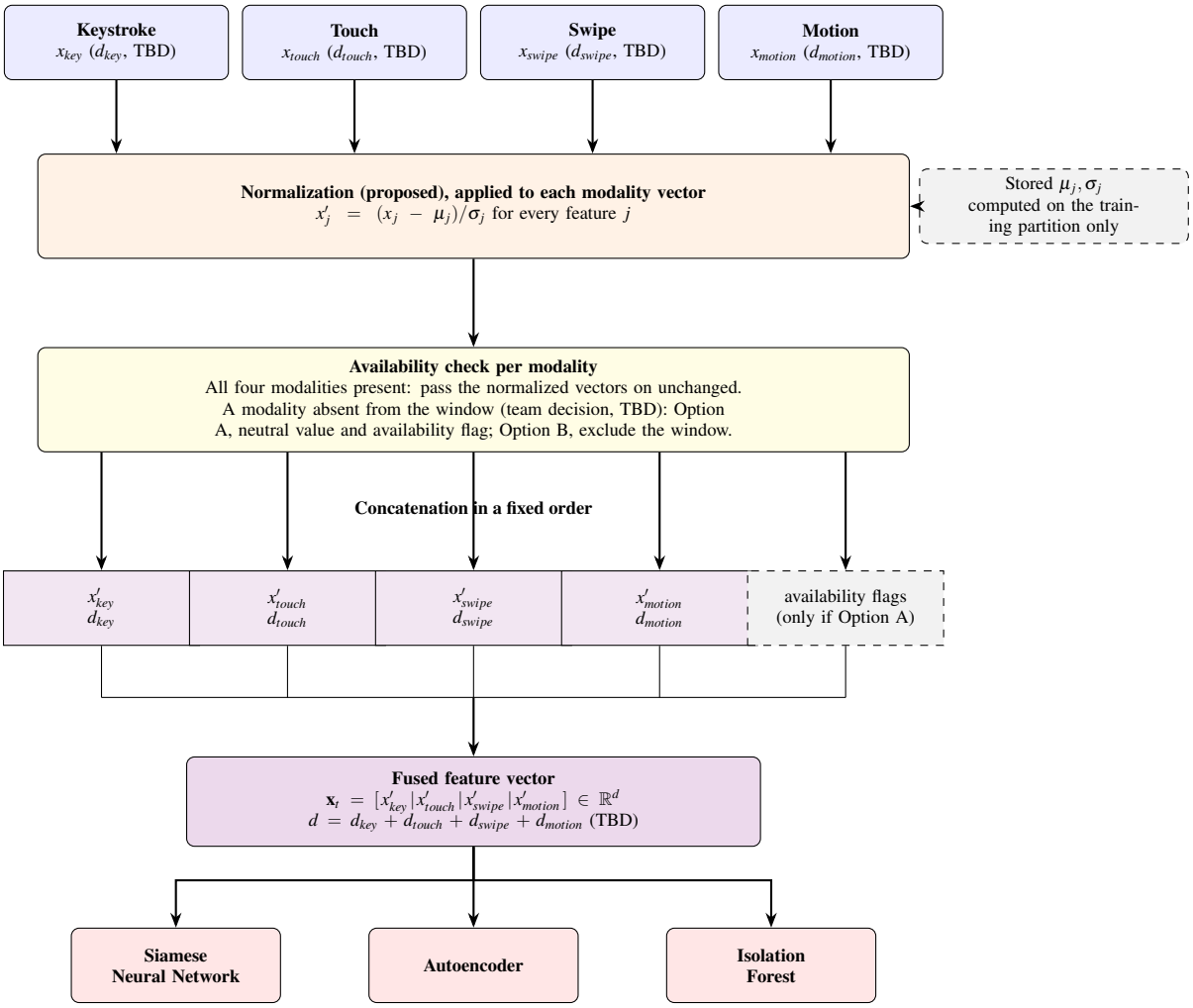


Figure 1.9. Normalization and feature-level fusion in detail: the four modality vectors of the same window are standardized with stored training statistics, checked for availability, concatenated in a fixed order and supplied unchanged to the three models.

Feature Dimensions

The exact dimensions have not yet been finalized and will be fixed after feature extraction and feature selection are implemented.

- Keystroke feature dimension d_{key}: TBD
- Touch feature dimension d_{touch}: TBD
- Swipe feature dimension d_{swipe}: TBD
- Motion feature dimension d_{motion}: TBD
- Fused feature vector dimension:

$$d = d_{\text{key}} + d_{\text{touch}} + d_{\text{swipe}} + d_{\text{motion}} = \text{TBD} \tag{1.8}$$

to be finalized after the feature-selection implementation.

Different modalities produce different numbers of raw values. Each modality is therefore first reduced to a fixed-length descriptor, and the descriptors are concatenated, so that every model receives a single vector of dimension d.

The handling of a window in which a modality is absent (for example, no typing while the user is only browsing

balances) is to be decided during implementation, for instance a masking value or an availability indicator.

Feature Normalization

A normalization method has not yet been finalized. The following approach is proposed: each feature is standardized to zero mean and unit variance (or alternatively min-max scaled), with the scaling parameters computed on the training partition only and then applied unchanged to validation, test and live data. This prevents data leakage.

Normalization is particularly important for the Autoencoder, because reconstruction error is scale-sensitive, and for the Siamese Network, because embedding distances depend on feature scale. The Isolation Forest is less sensitive to scale, but it uses the same normalized input for consistency.

Feature Selection

Feature selection and validation are carried out as part of the feature engineering work (Section 1.5). The selection algorithm has not yet been finalized. The ML models will use whichever final feature set is produced, which is why the fused dimension d is currently TBD.

1.6.3 Model Selection

Selected Approach

The proposed approach is a set of three complementary models: a Siamese Neural Network, an Autoencoder and an Isolation Forest. Each model produces its own score for the same behavioral feature vector, and the scores are combined into a single trust score. No single model is designated superior before experimental comparison. One-Class SVM, named in the project objectives, is retained as a baseline model for comparative evaluation and is not part of the proposed model set.

All three models can be built from the legitimate user's own behavior, together with data from other volunteers where the Siamese Network needs it. This matches the project requirement that attacker behavior is not available in advance.

Siamese Neural Network

Learning principle: Similarity (metric) learning. Two inputs pass through identical, weight-shared encoders, and the network is trained so that samples from the same user are close in the embedding space and samples from different users are far apart.

Input and output: The input is a pair of feature vectors. At inference, the current-window vector is compared with the enrolled reference samples of the claimed user. The output is a similarity (or distance) score.

Role in the system: It answers the question "does the current behavior resemble the enrolled user's behavior?" by comparing it directly with stored reference samples.

Why it is suitable: Because it learns a general similarity function rather than a fixed classifier for each user, a new user can be enrolled from a small number of samples. This suits a banking application in which new customers join continuously.

Strengths: Few-sample enrollment; a discriminative embedding space; a direct, interpretable similarity score.

Limitations: Training needs data from several users; the results depend on how sample pairs are formed and on the chosen loss; the score quality depends on the quality of the reference samples.

Autoencoder

Learning principle: Unsupervised reconstruction learning. The network compresses the input into a lower-dimensional latent representation and reconstructs it. It is trained only on the legitimate user's behavior.

Input and output: The input is the feature vector. The output is the reconstructed vector and its reconstruction error.

Role in the system: It answers the question "does the current behavior fit the pattern of normal behavior that the model has learned?" Behavior that differs from the training data is reconstructed poorly, so its error is higher.

Why it is suitable: It needs no attacker data, which matches the one-class nature of the problem, and it can capture nonlinear relationships among features from different modalities.

Strengths: Learns correlations between features; produces a continuous error score; trains on normal data only.

Limitations: It may reconstruct some unusual samples well if the capacity is too large; it is sensitive to feature scaling and bottleneck size; it needs sufficient normal data.

Isolation Forest

Learning principle: Tree-based, unsupervised outlier detection. Random partitioning of the feature space isolates unusual points in fewer splits than typical points.

Input and output: The input is the feature vector. The output is an anomaly score between 0 and 1.

Role in the system: It provides a fast, distribution-free check of whether the current behavior is an outlier relative to the legitimate user's feature distribution.

Why it is suitable: Scoring needs only a traversal of the trees, with no distance computation or gradient training, so it is lightweight. This is useful for the continuous, low-latency checks required in the application.

Strengths: Low computational cost; few assumptions about the data distribution; a simple, robust baseline.

Limitations: It splits on one feature at a time, so it may miss subtle deviations that arise from combinations of features; the score depends on the number of trees, sub-sample size and contamination setting.

Why Multiple Complementary Models Are Used

The three models examine the same behavior from different perspectives. The Siamese Network measures closeness to enrolled examples, the Autoencoder measures fit to the learned structure of normal behavior, and the Isolation Forest measures how rare the behavior is in the feature space.

A deviation that one model underestimates may be detected by another, so the combined trust score is expected to be more informative than any single score. This also supports the project's graded response (continue, step-up, terminate) instead of a binary decision. The relative performance of the models and the benefit of combining them will be established through the comparative evaluation (Section 1.6.7).

1.6.4 Model Architecture

Note: No layer counts, neuron counts, embedding or latent dimensions, or activation functions have been finalized. All such values are marked TBD or proposed and will be fixed during implementation and tuning.

Siamese Neural Network

Architecture flow:

Input A $\rightarrow$ Shared Encoder $\rightarrow$ Embedding A

Input B $\rightarrow$ Same Shared Encoder $\rightarrow$ Embedding B

Embedding A + Embedding B $\rightarrow$ Distance / Similarity $\rightarrow$ Similarity Score

The network consists of two branches that process two feature vectors, $\mathbf{x}_a$ and $\mathbf{x}_b$. Both branches are the same network f_θ applied twice with identical weights θ. Weight sharing is required for two reasons. First, it guarantees that both inputs are mapped by the same function into the same embedding space, so the distance between two embeddings is meaningful. Second, it halves the number of parameters and ensures that gradients from both branches update a single set of weights.

In the proposed system, $\mathbf{x}_a$ is the feature vector of the current session window and $\mathbf{x}_b$ is a stored reference sample of the legitimate user (or the reference samples are compared in turn). The shared encoder is a feed-forward network over the fused feature vector. Its depth, width and activation functions are TBD.

Embedding:

$$\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^k \tag{1.9}$$

where the embedding dimension k is TBD.

Distance and similarity (proposed):

$$D(\mathbf{x}_a, \mathbf{x}_b) = \|f_\theta(\mathbf{x}_a) - f_\theta(\mathbf{x}_b)\|_2 \tag{1.10}$$

This distance may be converted to a similarity such as:

$$s = \frac{1}{1+D} \tag{1.11}$$

or cosine similarity may be used instead. The final choice is to be made experimentally.

Loss (proposed, not finalized): Contrastive loss,

$$L = yD^2 + (1-y)\max(0, m-D)^2 \tag{1.12}$$

where $y = 1$ for a same-user pair, $y = 0$ for a different-user pair and m is the margin; or triplet loss,

$$L = \max(0, D(a,p) - D(a,n) + m) \tag{1.13}$$

for an anchor a, positive p and negative n. The margin m is TBD. The final choice will be made experimentally.

Use in authentication: The similarity score $s_{\text{sim}}(\mathbf{x}_t)$ is computed between the current window and the user's reference embeddings $\mathbf{r}_u$ (for example, the mean or maximum over the reference set). A high value supports the hypothesis that the behavior is genuine, and the score is passed to the Trust Score Generator.

Autoencoder

Architecture flow:

Input Feature Vector → Encoder → Latent Representation → Decoder → Reconstructed Feature Vector

Reconstructed Feature Vector → Reconstruction Error → Anomaly Score

- **Encoder h_ψ:** maps the input $\mathbf{x} \in \mathbb{R}^d$ to a lower-dimensional code.
- **Latent representation:**
$$\mathbf{z} = h_\psi(\mathbf{x}) \in \mathbb{R}^m \tag{1.14}$$
where the latent dimension m is TBD and is smaller than d, so the network cannot simply copy its input.
- **Decoder g_ϕ:** maps $\mathbf{z}$ back to the input space to give the reconstruction
$$\hat{\mathbf{x}} = g_\phi(\mathbf{z}) \in \mathbb{R}^d \tag{1.15}$$
- **Reconstruction loss for training (proposed):** mean squared error,
$$L = \frac{1}{N}\sum_{i=1}^{N} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 \tag{1.16}$$
- **Anomaly score for inference:**
$$s_{\text{ae}}(\mathbf{x}) = \|\mathbf{x} - \hat{\mathbf{x}}\|^2 \tag{1.17}$$

The network is trained only on the legitimate user's behavior (a one-class setting). It therefore learns to reconstruct normal behavior accurately, whereas behavior that differs from it is reconstructed poorly and gives a higher reconstruction error. The error is normalized and passed to the Trust Score Generator, where a larger error lowers the trust score. The number of layers, units and activation functions are TBD.

Isolation Forest

Isolation Forest is a traditional (non-neural) machine-learning model. Its pipeline is:

Input Features → Preprocessing / Normalization → Feature Selection (if applied, Section 1.6.2) → Isolation Forest → Anomaly Score

Inside the Isolation Forest the process is:

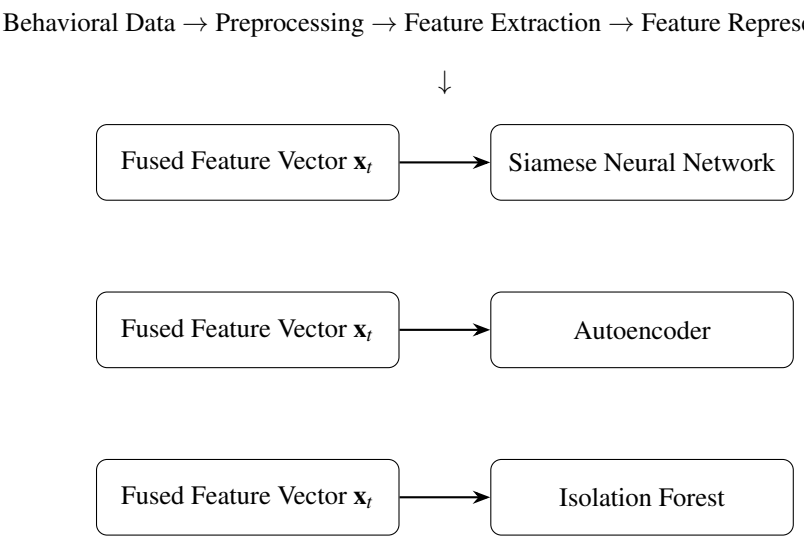
Feature Vector → Random Sampling / Feature Selection → Isolation Trees → Path Length Calculation → Aggregated Anomaly Score

- **Random partitioning:** each isolation tree is built from a random sub-sample of the training data. At every node, a feature is chosen at random and a split value is chosen at random between the minimum and maximum of that feature in the node. The partitioning continues until each point is isolated or a depth limit is reached.
- **Isolation trees:** the forest is an ensemble of T such trees.
- **Path length:** for an observation $\mathbf{x}$, $h(\mathbf{x})$ is the number of splits needed to isolate it in a tree. The average over the forest is $\mathbb{E}[h(\mathbf{x})]$.
- **Why anomalies are isolated faster:** behavior that deviates from the usual pattern lies in sparse regions of the feature space, so random splits separate it from the rest after few partitions (short path), whereas typical behavior lies in dense regions and needs many partitions (long path).
- **Anomaly score:**
$$s_{\text{if}}(\mathbf{x}) = 2^{-\frac{\mathbb{E}[h(\mathbf{x})]}{c(n)}} \tag{1.18}$$
where $c(n)$ is the average path length of an unsuccessful search in a binary search tree of n samples, used for normalization. Scores close to 1 indicate anomalies; scores well below 0.5 indicate normal behavior.

Isolation Forest is not a neural network and is not trained by gradient descent; it is fitted by building random trees. The score is passed to the Trust Score Generator, where a higher anomaly score lowers the trust score.

Overall ML Architecture and Score Fusion

Overall architecture flow:



Decision: continue / step-up authentication / terminate

Fusion terminology: Feature-level fusion of the four behavioral streams takes place in the Fusion stage before the models, as shown in the system architecture. The Trust Score Generator then combines the three model outputs into one trust score. The exact combination formula has not been finalized. The following scheme is proposed:

1. **Normalize each score to** $[0,1]$ using statistics obtained from validation data (for example, min-max scaling or percentile ranks), written $\mathrm{norm}(\cdot)$.
2. **Orient the scores so that a higher value always means more genuine:**

$$\hat{s}_{\mathrm{sim}} = \mathrm{norm}(s_{\mathrm{sim}}), \qquad \hat{s}_{\mathrm{ae}} = 1 - \mathrm{norm}(s_{\mathrm{ae}}), \qquad \hat{s}_{\mathrm{if}} = 1 - \mathrm{norm}(s_{\mathrm{if}}) \tag{1.19}$$

3. **Combine the scores with a weighted sum:**

$$T_t = w_{\mathrm{sim}}\hat{s}_{\mathrm{sim}} + w_{\mathrm{ae}}\hat{s}_{\mathrm{ae}} + w_{\mathrm{if}}\hat{s}_{\mathrm{if}} \tag{1.20}$$

with

$$w_{\mathrm{sim}} + w_{\mathrm{ae}} + w_{\mathrm{if}} = 1 \tag{1.21}$$

The weights are TBD and will be tuned on validation data. Alternatives such as learned or adaptive weighting may be compared.

4. **Smooth the score over time**, for example with an exponential moving average:

$$\widetilde{T}_t = \alpha T_t + (1-\alpha)\widetilde{T}_{t-1} \tag{1.22}$$

where α is TBD, so that a single noisy window does not trigger step-up authentication.

5. **Apply the decision rule** to $\widetilde{T}_t$ (or to T_t if smoothing is not used), using thresholds $\tau_1 > \tau_2$ (TBD) as defined in Section 1.6.1.

The feature-level pipeline is shown in Figure 1.7 and the complete system data flow in Figure 1.4.

1.6.5 Training Design

The complete pipeline, from data collection to the authentication decision, is shown in Figure 1.4. This figure is coordinated with the Data Flow / ML Pipeline diagram in Section 1.4.

Note: No training has been carried out yet, because the dataset is still being collected. All values below are proposed and will be finalized experimentally on the validation data. Numbers of users, sessions, samples and sample pairs are TBD.

General Training Setup

The partitioning of the data into training, validation and test sets, including the stratification strategy and the prevention of data leakage, is defined in Section 1.5.5. This section assumes the following:

- Normalization parameters are fitted on the training partition only and then applied unchanged to validation, test and live data.
- Samples from other volunteers are used to form the negative pairs of the Siamese Network and to evaluate impostor behavior. They are not used as attacker training data for the one-class models (Autoencoder and Isolation Forest) of a given user.
- The validation partition is used to select hyperparameters, to apply early stopping, and to calibrate the score normalization, the fusion weights and the thresholds τ_1 and τ_2. The test partition is used only for the final evaluation.

Any change to the splitting strategy in Section 1.5.5 should be reflected here.

Siamese Neural Network

Training procedure:

1. **Positive pairs:** two feature vectors from the same user, label $y = 1$.
2. **Negative pairs:** two feature vectors from two different users, label $y = 0$.
3. **Pair balancing:** positive and negative pairs are kept balanced, or the loss is weighted, so that the many easy negative pairs do not dominate training. The ratio is TBD.
4. **Forward pass:** both inputs of a pair pass through the same shared encoder f_θ and produce the embeddings $\mathbf{z}_a$ and $\mathbf{z}_b$.
5. **Distance or similarity:** $D(\mathbf{z}_a, \mathbf{z}_b)$ is computed (Euclidean distance proposed; cosine similarity as an alternative).
6. **Loss:** contrastive loss (or triplet loss), as defined in Section 1.6.4 (Equation 1.12). The final loss and the margin m are to be decided experimentally.
7. **Backpropagation:** the gradients from both branches are accumulated and update the single shared set of weights θ.
8. **Validation:** after each epoch, the validation loss and a verification measure on validation pairs (for example EER) are computed. Where possible, validation users are kept separate from training users to test generalization to new users.
9. **Checkpointing:** the weights with the best validation result are saved, and early stopping ends training when validation performance stops improving for a set number of epochs (patience TBD).
10. **Enrollment after training:** the trained encoder is applied to the legitimate user's enrollment samples to compute the reference embeddings $\mathbf{r}_u$. This step uses only the legitimate user's own data, so no attacker data is needed for that user.

Proposed settings: optimizer Adam; learning rate 0.001–0.0001; batch size 16–64; epochs 20–100 with early stopping; dropout 0.2–0.5; weight decay as additional regularization (value TBD).

Autoencoder

Training procedure:

1. **Training samples:** feature vectors of legitimate behavior only. Whether one model is trained per user or one model is trained on pooled legitimate data is to be decided (see Section 1.5.5).
2. **Forward pass:** the input $\mathbf{x}$ passes through the encoder to the latent representation $\mathbf{z}$, and the decoder produces the reconstruction $\hat{\mathbf{x}}$.
3. **Reconstruction loss:** mean squared error (proposed), as in Equation 1.16.
4. **Optimization:** gradients of the loss are computed by backpropagation and the weights of the encoder and decoder are updated by the optimizer.
5. **Validation:** the reconstruction loss is computed on held-out legitimate samples. Impostor samples are not used for training. They are used for calibrating the anomaly score and for evaluation.
6. **Early stopping and checkpointing:** training stops when the validation reconstruction loss stops improving (patience TBD), and the best weights are saved.
7. **Regularization:** a bottleneck smaller than the input ($m < d$), dropout and weight decay are used to prevent the network from learning a trivial copy of the input.

Proposed settings: optimizer Adam; learning rate 0.001–0.0001; batch size 16–64; epochs 20–100 with early stopping; dropout 0.2–0.5 (or none if the bottleneck alone is sufficient); weight decay TBD.

Isolation Forest

Isolation Forest is not trained with a loss function, an optimizer or backpropagation. It is fitted by constructing random trees.

Training procedure:

1. **Training vectors:** feature vectors of legitimate behavior.
2. **Sub-sampling:** each tree is built from a random sub-sample of the training vectors (size *max_samples*).
3. **Tree construction:** at every node a feature is chosen at random and a split value is chosen at random between that

feature's minimum and maximum in the node. The data are split recursively until each point is isolated or the depth limit is reached (about $\log_2$ of the sub-sample size).

4. **Path-length calculation:** for each observation, the number of splits needed to isolate it is recorded in every tree, and the average path length $\mathbb{E}[h(\mathbf{x})]$ over the forest is computed.
5. **Anomaly scoring:**

$$s_{\text{if}}(\mathbf{x}) = 2^{-\frac{\mathbb{E}[h(\mathbf{x})]}{c(n)}} \tag{1.23}$$

is computed from the average path length.

6. **Parameter selection:** the number of trees, sub-sample size, contamination and number of features per split (Table 1.15) are selected using the validation data, for example through the separation between legitimate and impostor scores. Epochs, learning rate, batch size and early stopping do not apply to this model.

1.6.6 Hyperparameter Design

The important hyperparameters of the three models are listed in Table 1.15, together with their proposed values or ranges and their purpose. The neural-network ranges follow the usual design practice for small feed-forward models. None of the values has been finalized experimentally. All of them are proposed and will be tuned on the validation partition defined in Section 1.5.5. Where a value is marked TBD, it depends on the final feature dimension d and on the size of the self-collected dataset, both of which are still to be finalized.

Justification of the proposed ranges: The dataset is self-collected from volunteer users and is expected to be modest in size. Small batch sizes, a modest learning rate, dropout and early stopping are therefore proposed to limit overfitting and to give stable convergence. The Isolation Forest parameters follow commonly used default ranges and will be adjusted using the separation between legitimate and impostor scores on validation data.

Tuning approach (proposed): Grid search or random search over the ranges in Table 1.15. The selection criterion is the validation EER for the Siamese Network and for the fused trust score, and the validation reconstruction loss for the Autoencoder. The final values will replace the ranges once the experiments are complete.

Table 1.15. Hyperparameter design (all values TBD).

Model	Parameter	Proposed Value/Range	Purpose
Siamese Neural Network	Optimizer	TBD	Adaptive gradient-based weight updates
Siamese Neural Network	Learning rate	TBD	Controls the size of each weight update
Siamese Neural Network	Batch size	TBD	Number of pairs processed per update
Siamese Neural Network	Epochs	TBD	Maximum number of training iterations
Siamese Neural Network	Loss function	TBD	Trains the embedding so that same-user samples are close and different-user samples are far
Siamese Neural Network	Margin m	TBD	Minimum required separation between different-user embeddings
Siamese Neural Network	Number of encoder layers	TBD	Controls model capacity
Siamese Neural Network	Hidden units per layer	TBD	Controls model capacity
Siamese Neural Network	Embedding dimension k	TBD	Size of the embedding (similarity) space
Siamese Neural Network	Activation function	TBD	Introduces non-linearity
Siamese Neural Network	Dropout	TBD	Reduces overfitting
Siamese Neural Network	Weight decay (L2)	TBD	Regularizes the weights
Siamese Neural Network	Distance function	TBD	Defines how embedding similarity is measured
Autoencoder	Optimizer	TBD	Adaptive gradient-based weight updates
Autoencoder	Learning rate	TBD	Controls the size of each weight update

Model	Parameter	Proposed Value/Range	Purpose
Autoencoder	Batch size	TBD	Number of samples processed per update
Autoencoder	Epochs	TBD	Maximum number of training iterations
Autoencoder	Reconstruction loss	TBD	Measures the difference between input and reconstruction
Autoencoder	Number of encoder/decoder layers	TBD	Controls model capacity
Autoencoder	Latent dimension m	TBD	Strength of the bottleneck; prevents copying of the input
Autoencoder	Activation function	TBD	Introduces non-linearity; allows reconstruction of standardized inputs
Autoencoder	Dropout	TBD	Reduces overfitting
Autoencoder	Weight decay (L2)	TBD	Regularizes the weights
Isolation Forest	`n_estimators`	TBD	Number of isolation trees; more trees give more stable scores
Isolation Forest	`max_samples`	TBD	Sub-sample size per tree; limits cost and reduces masking of anomalies
Isolation Forest	`contamination`	TBD	Expected proportion of outliers; sets the default decision cut-off if used
Isolation Forest	`max_features`	TBD	Fraction of features considered per split
Isolation Forest	`random_state`	TBD	Makes results reproducible
Fusion and decision (proposed)	Fusion weights $w_{\text{sim}}, w_{\text{ae}}, w_{\text{if}}$	TBD	Relative influence of each model in the trust score
Fusion and decision (proposed)	Smoothing factor α	TBD	Reduces the effect of a single noisy window on the trust score
Fusion and decision (proposed)	Thresholds $\tau_1 > \tau_2$	TBD	Boundaries for continue, step-up authentication and terminate

1.6.7 Model Evaluation Metrics

The evaluation uses authentication-specific measures together with standard classification metrics. The primary problem type is anomaly detection, but the final session decision is a thresholded mapping of the trust score, so classification-type metrics are meaningful at the decision stage. Regression metrics (MAE, MSE, RMSE, R^2), object-detection metrics (IoU, mAP) and clustering metrics (Silhouette score, Davies-Bouldin index) do not apply to this project.

Evaluation uses genuine samples (the legitimate user's behavior) and impostor samples (the behavior of other volunteers), taken from the test partition defined in Section 1.5.5 and reported separately for Objectives A and B. Each of the three models, and the fused trust score, will be evaluated under the same data and conditions, and compared with the One-Class SVM baseline. No evaluation has been carried out yet, and no results are reported in this document.

Authentication-Specific Metrics

False Acceptance Rate (FAR):

$$\text{FAR} = \frac{\text{number of impostor attempts accepted}}{\text{total impostor attempts}} \tag{1.24}$$

FAR shows how often unauthorized behavior is wrongly accepted. In mobile banking, a high FAR means an unauthorized person can continue a session and move money, so it measures security risk directly.

False Rejection Rate (FRR):

$$\text{FRR} = \frac{\text{number of genuine attempts rejected}}{\text{total genuine attempts}} \tag{1.25}$$

FRR shows how often the legitimate user is wrongly rejected. A high FRR causes unnecessary step-up authentication prompts or terminated sessions, so it measures usability and the cost of false alarms.

Equal Error Rate (EER):

EER is the error rate at the threshold where FAR equals FRR. FAR and FRR trade off against each other as the threshold changes, so EER gives one threshold-independent number for comparing the Siamese Network, the Autoencoder, the Isolation Forest, the One-Class SVM baseline and the fused trust score. It is the standard comparison measure in biometric authentication. The operating thresholds τ_1 and τ_2 used in deployment will be chosen from the FAR-FRR trade-off, with the higher cost of a missed impostor in banking taken into account.

Standard Classification Metrics

$$\text{Precision} = \frac{TP}{TP+FP} \tag{1.26}$$

$$\text{Recall} = \frac{TP}{TP+FN} \tag{1.27}$$

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{1.28}$$

Here an impostor (anomalous) sample is the positive class. Precision shows how many raised alarms are real impostor events, and recall shows how many impostor events are detected. F1-score combines the two into one value, which is useful because genuine and impostor samples are likely to be imbalanced.

ROC-AUC: The area under the ROC curve (true positive rate against false positive rate over all thresholds). It measures how well each raw score (similarity, reconstruction error, anomaly score, trust score) separates genuine from impostor behavior, independently of any chosen threshold. It is therefore useful for comparing the models before thresholds are set.

Confusion matrix: The counts of true positives, false positives, true negatives and false negatives at the chosen operating threshold. It shows which kind of error dominates, and relates directly to FAR and FRR.

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \tag{1.29}$$

Accuracy is reported as a secondary measure only. With imbalanced genuine and impostor data, a high accuracy can hide a poor detection of impostors, so it is not used as the main criterion.

Operational Metric

Authentication latency: The time from the arrival of a behavioral window to the update of the trust score (feature computation, model inference and fusion). The system runs continuously in the background of a banking session, so slow checks leave an unauthorized user acting for longer, and heavy computation affects the user's device. Latency will be measured for each model and for the complete pipeline, and is also relevant to the decision between on-device inference (TensorFlow Lite) and inference through the REST API.

Evaluation Protocol (proposed)

- Scores are computed for genuine and impostor test samples, and the metrics above are calculated per model and for the fused trust score.
- The results are reported for each of the three proposed models, for the fused trust score and for the One-Class SVM baseline.
- Thresholds and fusion weights are fixed using the validation partition only, and the test partition is used once for the final reported values.
- Data partitioning and leakage prevention follow Section 1.5.5.